

EcoTable: Cost-effective Table Integration in Data Lakes for Natural Language Queries

Yuhui Wang, Jinqi Liu, Chengliang Chai, Hangyu Zhao, Yuhao Deng,
Yuyu Luo, Xin Tang, Ye Yuan, Guoren Wang, Fengjin Wang*, Lei Cao[†]
University of Arizona[†], Kuaishou Technology*, Beijing Institute of Technology

Abstract

The diverse formats of CSV and Parquet files in data lakes pose a significant challenge to traditional ETL, which relies on data engineers to pre-define a target database schema and build a complex pipeline for data integration. Moreover, with this approach, the integrated data often cannot support various analytical needs, as the predefined schema does not necessarily satisfy the table format or join relationships required to answer unforeseen queries. To address this, we propose EcoTable, the first natural language-based data integration framework. Given a set of user-specified natural language queries, EcoTable automatically integrates the tables into a form that adequately supports the corresponding SQL queries. EcoTable achieves this by leveraging the semantic understanding and complex reasoning capabilities of Large Language Models (LLMs). Moreover, EcoTable addresses the scalability and cost issues introduced by expensive LLM inferences with a set of novel ideas.

EcoTable first introduces a graph to represent the overall search space, where nodes represent tables and edges carry weights indicating join likelihood produced by a lightweight deep learning model. On top of this graph data structure, EcoTable designs three components to achieve our goal: (1) the table identification layer aims to identify relevant tables via a two-stage schema linking based on user queries; (2) the graph-based validation layer aims to discover significant join paths, including necessary data transformations and bridging tables, by modeling the problem as Steiner tree searches; and (3) the table transformation layer generates transformation code to implement the joins using LLMs. We construct 4 real-world benchmark datasets with more than 200 queries. Extensive experiments demonstrate that EcoTable outperforms the state-of-the-art baselines, increasing accuracy by more than 30% and cutting LLM invocation costs by 5 times.

1 INTRODUCTION

Motivation. Data lakes store large collections of files (e.g., in CSV or Parquet formats) that do not fit neatly into relational tables, offering immense potential for data-driven decision making [3, 21]. However, deriving values from such a lightly structured data lake is notoriously difficult. The predominant approach relies on a traditional Extract-Transform-Load (ETL) process, where data engineers first *manually* define a target schema and then develop a complex pipeline to integrate data into a data warehouse [49, 64]. This is challenging. First, it is rather difficult to pre-define an appropriate database schema that meets the diverse analytical needs of an application. Second, even if the schema were defined correctly, it is labor intensive to convert the data lake tables into the desired relational table format.

Next, we use a real example to better illustrate this.

Example 1.1. As shown in Figure 1, consider 5 tables (i.e., T_1 : Order, T_2 : Revision History, T_3 : Service Types, T_4 : Workflow Steps, and T_5 : Service Pricing) sampled from a real-world E-Commerce data lake. A data engineer might naturally define a database schema over these tables, where T_1 .order_id joins with T_2 .id. However, the pre-defined schema does not necessarily satisfy the analytical need of users. For instance, suppose a user wants to check order status and thus issues a query Q_1 , “Display the daily tracking information for the order, including the name of the delivery service and workflow step details by date.” This query involves three tables (i.e., T_1 : Order, T_3 : Service Types, and T_4 : Workflow Steps). However, in the initial database schema, it is impossible to join these tables to answer this query because T_1 lacks explicit join conditions with either T_3 or T_4 . Furthermore, integrating tables requires significant manual efforts. To join T_1 , T_3 and T_4 for Q_1 , a data engineer must: (1) bridge T_1 and T_4 via T_2 with the join between T_2 and T_4 on the shared step_id attribute; and (2) transform T_3 by splitting service_type to enable join with T_4 on service_id. This process is complex, as it necessitates incorporating additional tables into join path and transforming tables. Moreover, the user might want to know more about service fees and thus submit another query Q_2 : “How are delivery services grouped by type, with their process steps and fees?”. To answer Q_2 , the data engineer must integrate T_5 : Service Pricing with T_3 and T_4 – a new data integration requirement. The key step is to split the service_plan column in T_5 (e.g., “Regular (Version 1)”) into distinct service_type and version columns. This prepares T_5 to be joinable with T_3 on service_type column.

This example shows that different queries have different requirements on the database schema. Therefore, pre-defining a database schema to meet users’ analytical needs tends to be impractical. Instead, data integration has to be tailored to user queries.

Targeting this problem, we propose to build an automatic table integration system, called EcoTable. With this system, users only have to provide a set of natural language (NL) queries to intuitively describe their analysis intention. The system will automatically discover a database schema that adequately supports these queries and then perform data integration accordingly.

Existing work has proposed techniques to automate data integration such as schema matching [22, 38, 42], entity resolution [19, 39, 46, 51, 59, 60], data transformation [43, 44, 67], discovery of joinable columns [12, 25], etc. However, all of these approaches rely on a pre-defined target schema. They do not solve the problem of automatically discovering an appropriate database schema based on NL queries and tables. On the other hand, a line of research [29, 37, 63, 64] focuses on transforming tables, e.g., spreadsheets or web tables, into relational formats. However, because the relational schema these methods automatically discover is blind to

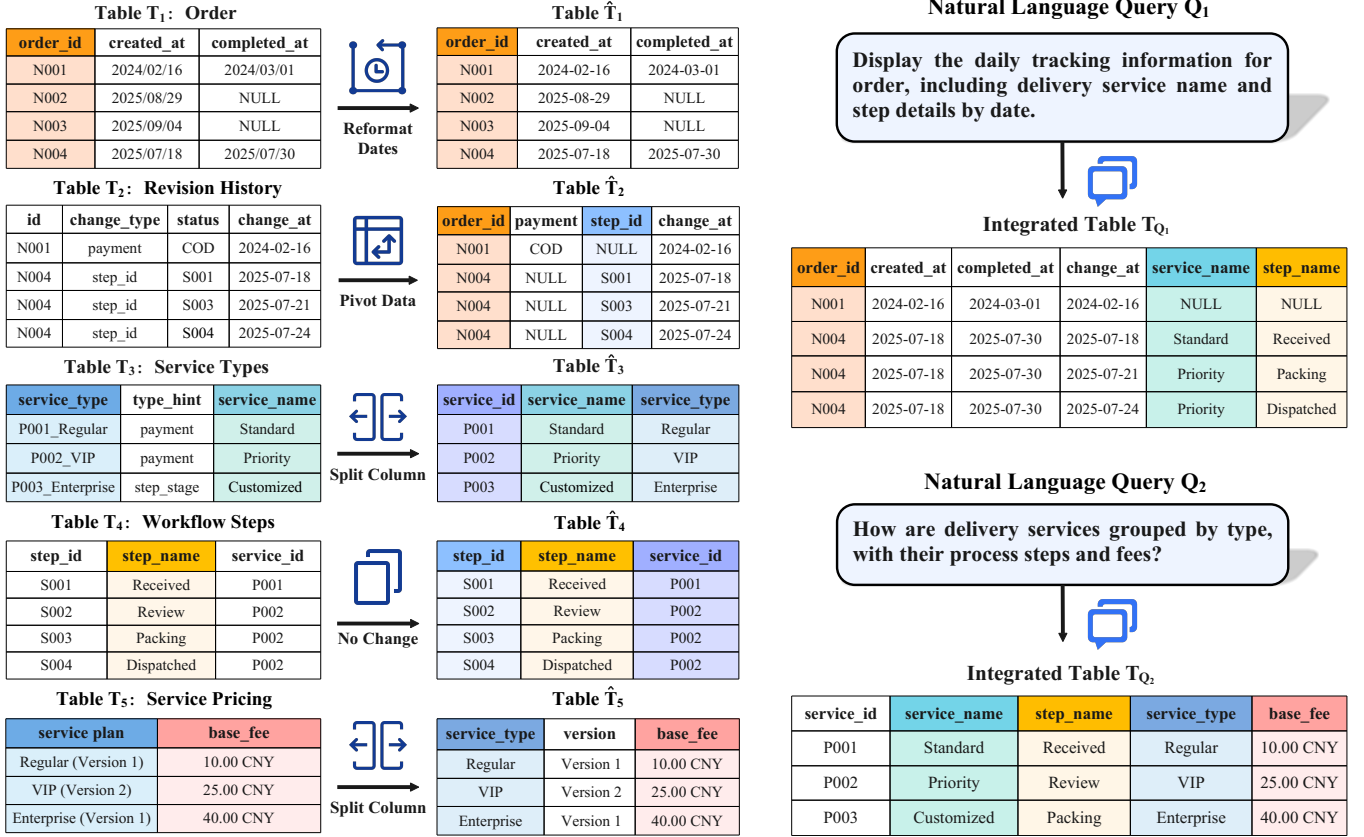


Figure 1: An Example of Query-driven Table Integration in Data Lakes.

user queries, it tends to be ineffective in supporting the analytical needs of users, as shown in the above example.

The semantic understanding and reasoning ability of Large Language Models (LLMs) make it possible for us to achieve the above goal. First, existing Text-to-SQL research has shown that LLMs excel at identifying the mapping between NL queries and columns in the tables through schema linking [4, 10, 54, 58]. Moreover, with its reasoning ability [32, 61, 62, 65], an LLM is able to produce a plan that combines multiple operations to collaboratively solve the data transformation problem.

Challenge. An intuitive method that leverages LLMs to solve this problem is to ask LLMs to (1) identify all tables that are relevant to the NL queries provided by users; (2) check whether every pair of these discover tables is joinable, thus finding a join path for each query; (3) finally, perform data transformation and integration accordingly. However, routinely invoking LLMs to explore all possible join relationships is prohibitively expensive. This is because the number of potential join paths grows exponentially with the number of tables. Therefore, the key challenge lies in how to minimize the number of LLM calls while still achieving high-quality integration to reliably answer user queries.

Our Proposal. To address this challenge, we propose EcoTable, a cost-effective query-driven table integration system. The key idea is to combine smaller deep learning models and LLMs to efficiently identify and validate potential join paths. At a high level, deep learning models give prior probabilities of possible join relationships

between tables. Then, to minimize LLM invocations, EcoTable prioritizes the pairs of tables for the LLMs to validate that are (1) most likely to be joinable and (2) required to answer user queries.

EcoTable models the search space as a fully connected graph where each node corresponds to a table, and each edge (i.e., the join relationship) connects two tables with a weight indicating the join likelihood initialized with a deep learning model. In this way, the problem of finding a *valid join path* for each NL query is converted to identify a connected subgraph that covers the tables involved in the query. EcoTable consists of three key components that efficiently discover valid join paths and perform data transformation on top of the graph.

Table Identification Layer identifies query-related tables (i.e., nodes) from all data lake tables via a two-stage schema linking method that first leverages a lightweight deep learning model for coarse-grained filtering, followed by using LLMs for fine-grained verification. This step effectively reduces the search space to a subset of nodes in the above graph, ensuring that subsequent steps primarily focus on these relevant tables.

Graph-based Validation Layer. Next, EcoTable discovers valid join paths to integrate these tables. As observed in the above example, joining these tables may require additional bridging tables, which is challenging to discover among a potentially large number of tables. To address this, we model the problem as a Steiner tree search problem, where the goal is to find the minimal-weight tree, i.e., a tree composed of edges with the highest join likelihood, which

connects all relevant tables for each query, potential via a set of bridging tables.

More specifically, EcoTable gathers edges from Steiner trees w.r.t. various queries and submits them to LLMs for batch verification. This opens the opportunity to leverage shared edges among queries to reduce LLM costs. Then EcoTable updates the edge weights of the graph based on verified results and iteratively searches the Steiner trees until the necessary edges of all queries have been validated. In this way, EcoTable selectively applies LLMs to validate the most promising join relationships and progressively discovers valid join paths with minimal LLM usage.

Table Transformation Layer. EcoTable then applies the necessary data transformation operations to perform joins and integrate tables. EcoTable uses LLMs to automatically generate transformation code for each valid join path, guided by a ReAct-style reasoning process that alternates between transformation and join validation. To reduce latency, EcoTable further employs a graph-based parallelization strategy, which identifies parallelizable transformations by modeling the problem as an edge coloring task on the validated join paths.

In summary, we make the following contributions.

- (1) We propose EcoTable, to the best of our knowledge, the first natural language driven system for table integration that minimizes LLM invocations while adequately supporting user queries.
- (2) We design a graph-based validation layer that models table relationships as a weighted graph and identifies valid join paths using a Steiner tree-based optimization.
- (3) We introduce a table transformation layer that employs a ReAct-style reasoning loop to automatically generate transformation code for joinable tables and a graph-based parallelization strategy to further reduce latency and LLM invocation cost.
- (4) We construct 4 real-world benchmark datasets covering multiple domains with more than 200 natural language queries. Comprehensive experiments show that EcoTable surpasses state-of-the-art baselines by achieving over 30% accuracy improvement and reducing LLM invocation costs by 5 times.

2 PROBLEM DEFINITION

Given a data lake including a large number of tables (e.g., CSV or parquet files) and a set of user-specified natural language queries, we aim to integrate subsets of tables related to these queries such that each query could be well addressed by an integrated table. Due to the heterogeneity of the data lake tables, the integration process might be rather complicated, including transformations and joins over multiple tables. Formally, we define the problem as follows.

Definition 2.1 (Query-Driven Data Integration). Given N tables in a data lake $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$ and a set of M natural language queries $\mathcal{Q} = \{Q_1, Q_2, \dots, Q_M\}$, we aim to construct a set of transformed tables $\mathcal{T}^* = \{\hat{T}_1, \hat{T}_2, \dots, \hat{T}_n\}$ from $\mathcal{T}$ to form a global join graph $\mathcal{G}^* = (\mathcal{T}^*, E^*)$, such that each query $Q_k \in \mathcal{Q}$ can be successfully answered by querying a connected subgraph $\mathcal{G}_k^* = (\mathcal{T}_k^*, E_k^*)$ of $\mathcal{G}^*$, where $\mathcal{T}^* = \bigcup_{k=1}^M \mathcal{T}_k^*$ and $E^* = \bigcup_{k=1}^M E_k^*$, $k \in [1, M]$. That is, each connected subgraph $\mathcal{G}_k^*$ corresponds to an integrated table sufficient for answering query Q_k and the table is obtained by integrating tables in $\mathcal{T}_k^*$. The nodes and edges are defined as follows:

Nodes: For each query Q_k , the node set $\mathcal{T}_k^* = \{\hat{T}_k^1, \hat{T}_k^2, \dots\}$ contains the transformed tables required to answer the query. Each $\hat{T}_k^i \in \mathcal{T}_k^*$ is derived from a source table $T_k^i \in \mathcal{T}_k \subseteq \mathcal{T}$ through the transformation ϕ_k^i , i.e., $\hat{T}_k^i = \phi_k^i(T_k^i)$. In this paper, $\phi_k^i(\cdot)$ includes operations such as reformatting, splitting, and pivoting.

Edges: For any edge $\hat{e}_k^{ij} \in E_k^*$, it connects two tables $\hat{T}_k^i, \hat{T}_k^j \in \mathcal{T}_k^*$ if they can be joined to answer Q_k . Note that $\mathcal{G}_k^*$ is connected, corresponding to the join path, but $\mathcal{G}^*$ might not be when different queries are associated with totally different tables.

Example 2.2. Based on Example 1.1 and Figure 1, the mapping to the formal graph $\mathcal{G}^*$ is as follows:

- Query Q_1 is addressed by subgraph $\mathcal{G}_1^*$, with nodes $\mathcal{T}_1^* = \{\hat{T}_1, \hat{T}_2, \hat{T}_3, \hat{T}_4\}$ and edges $E_1^* = \{\hat{e}_1^{12}, \hat{e}_1^{24}, \hat{e}_1^{34}\}$. Each $\hat{T}_i$ is the result of the transformation ϕ_1^i (e.g., date reformatting for T_1 , table pivoting for T_2 , and column splitting for T_3).
- Query Q_2 is addressed by subgraph $\mathcal{G}_2^*$, which involves nodes $\mathcal{T}_2^* = \{\hat{T}_3, \hat{T}_4, \hat{T}_5\}$ and edges $E_2^* = \{\hat{e}_2^{12}, \hat{e}_2^{13}\}$. Here, $\hat{T}_5$ is the result of the transformation ϕ_2^3 (e.g., column splitting for T_5).
- The global join graph $\mathcal{G}^* = (\mathcal{T}^*, E^*)$ is defined by the unions $\mathcal{T}^* = \bigcup_{k=1}^M \mathcal{T}_k^*$ and $E^* = \bigcup_{k=1}^M E_k^*$, forming a unified structure that supports all analytical requirements.

Table Transformation resolves structural and semantic discrepancies in data lakes. We focus on three essential operators:

Normalize standardizes data formats for consistency, e.g., converting the created_at date in Table T_1 from "2024/02/16" to the ISO standard "2024-02-16" in Table $\hat{T}_1$.

Split decomposes composite columns into atomic ones, e.g., splitting the service_type column in Table T_5 (e.g., "Regular (Version 1)") into separate service_type and version columns in Table $\hat{T}_5$.

Pivot restructures tables by transposing unique values into new columns, e.g., applying *Pivot* to T_2 on the change_type column generates new column headers payment and step_id in Table $\hat{T}_2$.

Join Discovery identifies semantically equivalent columns across tables to form valid join paths. Existing approaches include:

Deep learning methods (e.g., DeepJoin [12] and OmniMatch [28]) encode column profiles into vector representations and predict the join likelihood between columns using PLMs. For example, T_2 .change_type and T_3 .type_hint might be considered as joinable because PLM-based methods focus more on token overlaps or string similarities. In reality, they refer to different things: change_type is a revision category, whereas type_hint is an interface tag (e.g., step_stage) denoting the record's current workflow stage, so they should not be joined.

LLM-based methods leverage fine-tuning [38] or prompt engineering [42] to understand table semantics and validate join relationships. These methods demonstrate strong few-shot learning capabilities and can handle complex semantic matching scenarios that are challenging for traditional approaches. By reasoning over column descriptions and data patterns, LLMs achieve robust performance even on unseen schema or ambiguous relationships.

Given $\mathcal{T}$ and $\mathcal{Q}$, a straightforward way to construct $\mathcal{G}^*$ is that for each query Q_k , we enumerate a large number of possible table combinations from $\mathcal{T}$ and identify the combination that can satisfy

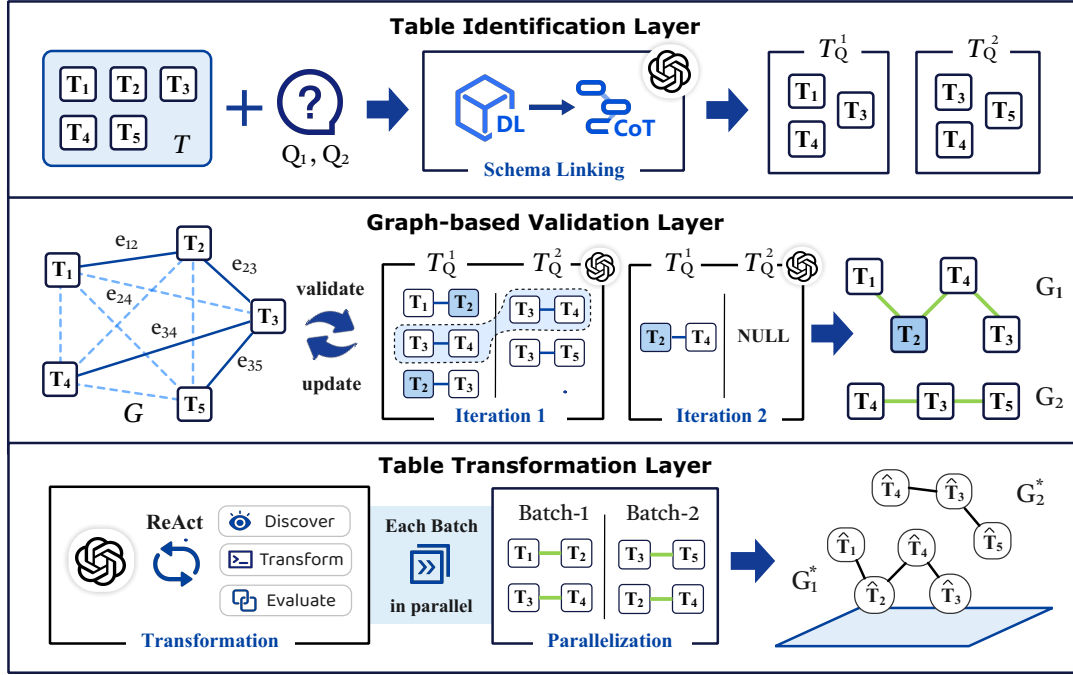


Figure 2: The Overall Framework of EcoTable.

the query, which is prohibitively expensive if we have to call LLMs to verify whether each combination is valid. Thus, in this paper, we propose the EcoTable framework to largely reduce the LLMs costs.

3 OVERVIEW

As illustrated in Sec. 1, EcoTable formulates the query-driven table integration task as a join path identification and validation problem over a graph, ensuring a balance between integration effectiveness and LLM invocation cost. Specifically, EcoTable first constructs a fully connected graph $\mathcal{G} = (\mathcal{T}, E)$. Then, EcoTable leverages the following three layers to integrate tables for correctly answering the given queries. As shown in Figure 2, the first table identification layer takes the queries as input, and outputs the table subsets relevant to these queries. Second, the graph-based validation layer takes these tables as input, and outputs valid join paths through searching within the graph $\mathcal{G}$, so as to connect these relevant tables. Third, the table transformation layer takes as input these identified join paths, applies necessary data transformation operations to actually join each two tables and outputs the integrated tables.

3.1 Table Identification Layer

In Figure 2, for example, given two queries Q_1 and Q_2 , the module applies schema linking to obtain the corresponding relevant table sets $\mathcal{T}_Q^1 = \{T_1, T_3, T_4\}$ and $\mathcal{T}_Q^2 = \{T_3, T_4, T_5\}$, a well-studied process [4, 31, 48]. To strike a balance between accuracy and LLMs cost for identifying these tables, we adopt a hybrid approach of combining LLMs and deep learning models. Specifically, we first train a deep learning model M_s to predict the relevance between an NL query and a table. This model takes as input the concatenation of the query and the table’s column names (i.e., its schema), embeds them and outputs a relevance score. Tables with high scores are selected

as candidates for subsequent verification by LLMs (see Sec. 4.1 for details).

3.2 Graph-based Validation Layer

As shown in Figure 2, given the identified table subsets $\mathcal{T}_Q^1$ and $\mathcal{T}_Q^2$, this module finds valid join paths $\mathcal{G}_1$ and $\mathcal{G}_2$ that connect them. As outlined in Sec. 1, identifying such join paths is challenging because: (1) making relevant tables joinable often requires appropriate data transformations or added bridge tables, and (2) the search space is large to identify the valid join path.

To address this, we formulate it as a Steiner tree search problem based on the fully connected graph $\mathcal{G}$. Initially, EcoTable uses a deep learning model M_j to estimate the likelihood of joining pairs of tables and assign these values as edge weights in the graph. After that, for each Q_k , EcoTable attempts to search a corresponding Steiner tree from $\mathcal{G}$, where (1) the nodes cover the tables in $\mathcal{T}_Q^k$ and (2) the edges have the maximum overall joinable probability. Naturally, this Steiner tree represents the most promising join path in which the tables in $\mathcal{T}_Q^k$ could be joined to answer Q_k (e.g., for $\mathcal{T}_Q^1 = \{T_1, T_3, T_4\}$, a Steiner tree introduces T_2 as a bridge with edges $\{e_{12}, e_{23}, e_{34}\}$, as shown in Figure 2).

However, deep learning is typically not accurate enough, so we collect all edges from the Steiner trees corresponding to different queries and employ LLMs for further verification. This processing leverages the fact that multiple queries may share common edges, thereby saving LLMs costs (e.g., edge e_{34} appears in both $\mathcal{T}_Q^1$ and $\mathcal{T}_Q^2$ ’s Steiner trees and requires only LLM validation once). Afterwards, we update the graph by removing edges that are deemed unjoinable by LLMs, while assigning a weight of one to those edges that are confirmed as joinable. We then iteratively recompute the Steiner trees using the updated graph and iterate the validation

process until for every query, we either find a join path that makes the query executable or conclude that no such path exists due to insufficient connectivity (see Section 4.2 for details).

3.3 Table Transformation Layer

Finally, in Figure 2, starting from the valid join paths $\mathcal{G}_1$ and $\mathcal{G}_2$, the module applies data transformation operations to get $\mathcal{G}_1^*$ and $\mathcal{G}_2^*$, the integrated tables used to answer queries Q_1 and Q_2 . To be specific, for each valid edge, EcoTable uses a transformation modular component M_T to automatically produce executable code, defined in a formal transformation grammar, for the two tables connected by the edge. To improve accuracy, LLMs verify each transformation using a ReAct-style reasoning process. For all these edges, a straightforward method is to batch multiple tables and ask LLMs to transform them together. However, this would degrade the accuracy due to complex chain dependencies within these tables, which is difficult for LLMs to understand. An alternative is to sequentially process each edge. However, this would incur high latency due to the iterative process of ReAct. To tackle this issue, EcoTable further introduces a graph-based parallelization strategy to identify independent transformations that can be executed concurrently based on the edge-coloring framework. The key observation is that two edges can be transformed in parallel if they do not share the same table. For example, in $\mathcal{G}_1$, transformations for e_{12} (T_1 - T_2) and e_{34} (T_3 - T_4) can be processed in parallel (see Section 5 for details).

4 GRAPH-BASED TABLE INTEGRATION

In this section, we introduce in details how EcoTable identifies query-related tables (Section 4.1), searches valid join paths (Section 4.2) and leverages LLMs to verify promising join relationships to produce finally integrated tables (Section 4.2).

4.1 Table Identification Layer

Stage 1 (PLM Table-level Selection). We fine-tune a lightweight-compact pre-trained language model, M_S , as a cross-encoder to compute relevance scores between a query and a table schema. Unlike bi-encoder setups that embed queries and schemas separately, M_S encodes them together in one sequence, allowing full-attention interactions between the query and schema metadata. As a modular component, M_S can be instantiated flexibly with various pre-trained backbones; in our implementation, we specifically support RoBERTa [41], RoBERTa-Large [23], and DeBERTa-v3-Large [20]. Subsequently, for each query, i.e., $Q_k \in \mathcal{Q}$, we choose the tables with large relevance scores as candidates, denoted as $\mathcal{T}_{\text{cand}}^k$. To be specific, we train the model on inputs of the form “*query* [TAB] *table_name* [COL] *col_1* [COL] *col_2* [COL], ...”. The training process optimizes the model to predict binary labels $y_i \in \{0, 1\}$ by minimizing cross-entropy loss:

$$\mathcal{L} = -[y_i \log s_i + (1 - y_i) \log(1 - s_i)] \quad (1)$$

where s_i denotes the relevance score predicted by the model.

Overall, this step retrieves a sufficient number of tables that may answer the queries, i.e., ensuring a high recall. However, exploring the join pairs among this relatively large number of tables is still expensive.

Stage 2 (LLMs Refinement). To address this, we employ LLMs to refine those candidate tables. Given $\mathcal{T}_{\text{cand}}^k$, EcoTable performs a fine-grained verification using the CoT reasoning capability of LLMs. Specifically, EcoTable prompts the LLMs to reason through the following steps:

Query Understanding. We utilize the LLMs as a query decomposer following a one-shot prompt setting. Given a natural language query Q , the LLMs are guided to split it into a sequence of sub-questions $q_1, q_2, \dots, q_n$, effectively simplifying the complex request into atomic semantic components such as entities and attributes. For example, for Q_1 in Figure 1, LLMs identify key entities as “*order*” and “*delivery service*”, attributes such as “*order_id*”, “*service_name*”, and “*step_name*”. This decomposition captures what the query focuses on.

Schema Reasoning and Alignment. For $\mathcal{T}_{\text{cand}}^k$, the LLMs use Chain-of-Thought to evaluate each candidate table step by step. Specifically, for each decomposed semantic component, the LLM tries to find a matching column in the table. It explicitly explains its reasoning, such as why a column matches a sub-question or why a table is missing the required information. If a candidate table cannot match any necessary components, the LLM rejects it. For example, for Q_1 , LLMs reason that the entity “*order*” from the query is represented by the table T_1 : Order, while its attribute “*order_id*” corresponds to the column *order_id* in T_1 . In this way, LLMs reliably identify the final verified table set $\mathcal{T}_Q^k \subseteq \mathcal{T}_{\text{cand}}^k$, which contains the tables that are necessary to answer the query.

4.2 Graph-based Validation Layer

Given the fully-connected global join graph $\mathcal{G} = (\mathcal{T}, E)$, each edge $e_{ij} \in E$ represents a potential join relationship between tables T_i and T_j , associated with a weight that represents the joinable probability $p(e_{ij}) \in [0, 1]$ computed by M_J . Given queries $\mathcal{Q} = \{Q_1, Q_2, \dots, Q_M\}$, EcoTable discover a collection of query-specific join paths $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_M\}$, where each $\mathcal{G}_k = (\mathcal{T}_k, E_k)$ connects tables in $\mathcal{T}_Q^k$ required to answer Q_k . Formally, this join path discovery problem can be formulated as:

$$\begin{aligned} \max_{\{x_{ij}^k\}} \quad & \sum_{k=1}^M \prod_{e_{ij} \in E} p(e_{ij})^{x_{ij}^k} \\ \text{s.t.} \quad & \forall k, \forall T_i \in \mathcal{T}_Q^k : \sum_{e_{ij} \in \delta(T_i)} x_{ij}^k \geq 1, \\ & \forall k, \forall S \subset \mathcal{T}_k', S \neq \emptyset, S \neq \mathcal{T}_k' : \sum_{e_{ij} \in \phi(S)} x_{ij}^k \geq 1. \end{aligned} \quad (2)$$

where $x_{ij}^k \in \{0, 1\}$ is a binary decision variable indicating whether edge e_{ij} is included in the join path of query Q_k . $\delta(T_i)$ denotes the set of edges corresponding to node T_i . $\mathcal{T}_k' = \{T_i | \exists T_j, x_{ij}^k = 1\}$, representing the set of tables (including the bridging tables) selected in the join path for query Q_k . For a subset $S \subseteq \mathcal{T}_k'$, $\phi(S)$ denotes the cut set of S (i.e., the edges connecting S to its complement $\mathcal{T}_k' \setminus S$), defined as $\phi(S) = \{e_{ij} \in E \mid T_i \in S, T_j \in \mathcal{T}_k' \setminus S\}$. The first constraint ensures that the join path includes all tables in $\mathcal{T}_Q^k$ for each query Q_k , while the second enforces connectivity. This optimization maximizes the sum of the overall joinability of each

Algorithm 1: Graph-based Validation Algorithm

Input: Global weighted join graph $\mathcal{G} = (\mathcal{T}, E)$, relevant tables for each query $\{\mathcal{T}_Q^1, \mathcal{T}_Q^2, \dots, \mathcal{T}_Q^M\}$.

Output: Validated join paths $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_M\}$.

```
1  $E^+ \leftarrow \emptyset, E^- \leftarrow \emptyset;$ 
2 while  $\neg \bigcup_1^k E_k \subseteq E^+ \wedge \text{HasTree}(\mathcal{G}, \{\mathcal{T}_Q^1, \dots, \mathcal{T}_Q^M\})$  do
3   // Step 1: Graph update
4    $\mathcal{G}_u \leftarrow \text{UpdateGraph}(\mathcal{G}, E^+, E^-);$ 
5   // Steps 2: LLMs validation
6    $E_b \leftarrow \emptyset;$ 
7   for  $k \in [1..M]$  do
8      $\mathcal{G}_k, \mathcal{T}_k, E_k = \text{SteinerTree}(\mathcal{G}_u, \mathcal{T}_Q^k);$ 
9      $E_b \leftarrow E_b \cup E_k;$ 
10   $E_{\text{new}} \leftarrow E_b \setminus (E^+ \cup E^-);$ 
11  for each  $e_{ij} \in E_{\text{new}}$  do
12    if  $\text{LLMValidate}(e_{ij}) = \text{VALID}$  then
13       $E^+ = E^+ \cup \{e_{ij}\};$ 
14    else
15       $E^- = E^- \cup \{e_{ij}\};$ 
16 for  $k \in [1..M]$  do
17    $\mathcal{G}_k = (\mathcal{T}_k, E^+ \cap E_k);$ 
18 return  $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_M\};$ 
```

query, which is computed by multiplying the join likelihoods of edges in the query’s join path, i.e., $\prod_{e_{ij} \in E} p(e_{ij})^{x_{ij}^k}$. In this way, we prioritize the edges that are likely to be joined to answer queries for LLMs to validate.

Pairwise Joinability Estimation. Following [40], EcoTable adopts a deep learning model M_J to compute $p(e_{ij})$. This model serves as an efficient but approximate estimator of joinability, providing a prior probability for each edge. Specifically, given two columns, M_J takes input table features (e.g., metadata and column embeddings) and outputs join probabilities in $(0, 1]$. Designed as a modular component, M_J allows for flexible instantiation. In this work, we utilize three models: Auto-BI [40], DeepJoin [12], and OmniMatch [28]. Formally, let $C \in T_i$ and $C' \in T_j$ denote two columns in tables T_i and T_j . $M_J(C_i, C_j) \in (0, 1]$ predicts the probability that C and C' could be joined. Because the joinability between two tables often depends on the presence of at least one pair of aligned columns, we use the maximum pairwise similarity as a joinability score and assign this as the weight, i.e., $p(e_{ij}) = \max_{C \in T_i, C' \in T_j} M_J(C, C')$.

Iterative Validation with LLMs. Based on the weighted join graph $\mathcal{G}$, EcoTable utilizes the typical algorithm [27] with an approximation ratio of 2 (i.e., the KMB algorithm) to efficiently identify a Steiner tree for each query Q_k . Then EcoTable performs iterative validation with LLMs to refine the accuracy of these discovered Steiner trees. Specifically, at the beginning of each iteration, EcoTable selects all unverified edges from the Steiner tree of each query. These edges are submitted to LLMs to validate join relationship. EcoTable updates the weight of the join graph $\mathcal{G} = (\mathcal{T}, E)$ accordingly as follows:

- If an edge e_{ij} passes validation, its corresponding weight is set to 1, ensuring that EcoTable will only validate one edge at most once.
- If e_{ij} is invalid, the edge is removed from the graph and will not be considered in later iterations.

For the updated graph, EcoTable recomputes a maximum Steiner tree for each query Q_k and checks the unverified edges in these new trees. This process repeats until all edges of the obtained trees pass the verification, or no more feasible Steiner trees are available. *Remark.* Since multiple queries often involve overlapping table pairs, EcoTable caches all validated results globally. If an edge has been validated once—either as valid or invalid—the result is reused directly in the Steiner tree searches of other queries, eliminating redundant LLM calls throughout the query set Q .

Algorithm Overview. Algorithm 1 summarizes the whole process of the graph-based validation layer. EcoTable initializes two edge sets E^+ and E^- to record valid and invalid joins verified before (line 1). In each iteration (line 2), EcoTable updates the join graph $\mathcal{G}$ by removing invalid edges and setting the weight of valid ones as 1 (line 4). It then searches for a new Steiner tree for each query over the updated graph (line 7-8). Next, the algorithm collects and deduplicates all unverified edges from these new Steiner trees into a candidate set E_{new} (line 9-10). Each edge is checked by LLMs. EcoTable adds valid edges to E^+ and invalid ones to E^- (line 11-15). It then uses the updated graph to recompute new Steiner trees for all queries until all edges pass the validation or no feasible tree remains. Finally, the algorithm outputs the validated join paths (line 16-18).

Remark. Because LLMs validation is prone to hallucinations, EcoTable further leverages majority voting for two types of critical edges: (1) edges that appear in Steiner trees corresponding to multiple queries within the same iteration, and (2) edges whose deletion would prevent constructing a new Steiner tree for a query in the subsequent iteration. EcoTable makes three independent LLM calls for each such edge and then adopts the majority decision.

Example 4.1. As shown in Figure 3, at the beginning, EcoTable constructs the initial weighted graph $\mathcal{G} = (\mathcal{T}, E)$, where the edge weights represent the joinability score estimated by model M_J . For each query, EcoTable first searches a Steiner tree that connects all query-relevant tables with the maximum total joinability score. The initial Steiner tree for Q_1 includes edges $\{e_{12}, e_{23}, e_{34}\}$, while for Q_2 , it covers edges $\{e_{34}, e_{35}\}$. In the first iteration, EcoTable validates all unverified edges of both trees using LLM reasoning. Since e_{34} appears in both trees, EcoTable makes three independent LLM calls for it. Following the majority decision, e_{34} is confirmed as valid. Simultaneously, e_{12} and e_{35} are verified, while e_{23} is rejected and removed. All results are stored in the global cache. The graph $\mathcal{G}$ is updated accordingly: valid edges are set to weight 1, while invalid edges such as e_{23} are removed. At this point, Q_2 already has a validated join path: $\mathcal{G}_2 = \{\{T_3, T_4, T_5\}, \{e_{34}, e_{35}\}\}$. In the second iteration, EcoTable recomputes the Steiner tree for Q_1 on the updated graph. The new candidate tree contains edges $\{e_{12}, e_{24}, e_{34}\}$. Since e_{12} and e_{34} are already cached as valid, only e_{24} is newly verified. LLMs confirm e_{24} as valid, completing the join path for Q_1 : $\mathcal{G}_1 = \{\{T_1, T_2, T_3, T_4\}, \{e_{12}, e_{24}, e_{34}\}\}$.

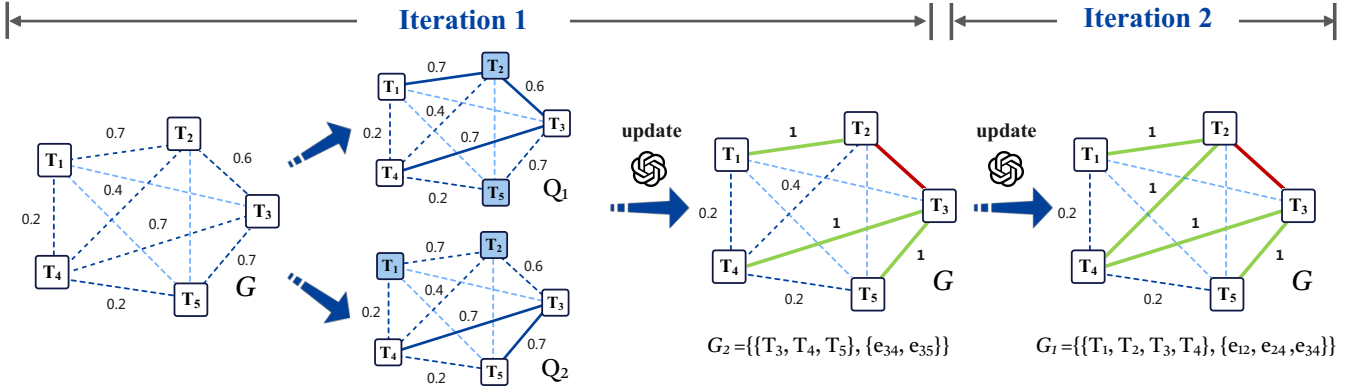


Figure 3: An Example of Join Path Search and Validation.

Edge pruning strategy. However, Algorithm 1 can exhibit quadratic time complexity $O(|\mathcal{T}|^2)$ because its search space may grow without bound. In particular, as long as the nodes in $\mathcal{T}_Q^k$ remain unconnected by valid edges, EcoTable must iteratively search for additional edges and, in the worst case, which will ultimately perform an exhaustive traversal of all edges to identify a feasible path. Consequently, in a data lake containing thousands of tables, this may require LLM calls on the order of all edges in the global graph, rendering both execution time and cost impractical. To tackle this problem, we introduce an edge-pruning approach that selects a subgraph $\mathcal{G}_s = (\mathcal{T}, E_s)$ whose validation performance theoretically approximates that of the full graph $\mathcal{G}$, while restricting the search to a substantially reduced exploration space.

The key property of the pruned subgraph $\mathcal{G}_s$ is to precisely retain high-probability join paths relevant to the given query because these paths are likely to cover the edges in Steiner Trees. In this way, we can still identify the same Steiner Trees in the subgraph with much lower LLMs costs. The key of computing a Steiner Tree for query Q_k is to calculate the path with maximum overall joinability (i.e., $\prod_{e_{ij} \in \pi} p(e_{ij})$) in $\mathcal{G}$ between each pair of tables in $\mathcal{T}_Q^k$. Therefore, the subgraph $\mathcal{G}_s$ only needs to include edges on those maximum-joinability paths between all table pairs in $\mathcal{T}_Q^k$. To this end, we leverage the t-spanner algorithm [5] to selectively prune edges in $\mathcal{G}$ that cannot be in the Steiner Tree for any query. The algorithm guarantees that for any table pair (u, v) , the maximum joinability of their paths in $\mathcal{G}_s$ is at least the t -th power of that in $\mathcal{G}$. Moreover, because the joinable probability p may not perfectly reflect the actual joinability, we additionally employ an f -path fault-tolerant mechanism [6] that preserves, for each pair of tables, $f + 1$ paths with the high joinabilities.

Formally, for any (u, v) in $\mathcal{T}_Q^k$, let $\mathcal{P}_{\mathcal{G}_s}(u, v)$ and $\mathcal{P}_{\mathcal{G}}(u, v)$ denote the set of all paths between u and v in $\mathcal{G}_s$ and $\mathcal{G}$, respectively. Let $F \subset \mathcal{P}_{\mathcal{G}}(u, v)$ be an arbitrary subset of paths with $|F| \leq f$, representing the situation where as many as f candidate paths are removed (for example, because they include edges that do not pass LLM verification). Then, the subgraph $\mathcal{G}_s$ satisfies:

$$\max_{\pi \in \mathcal{P}_{\mathcal{G}_s}(u, v) \setminus F} \prod_{e_{ij} \in \pi} p(e_{ij}) \geq \left(\max_{\pi^* \in \mathcal{P}_{\mathcal{G}}(u, v) \setminus F} \prod_{e_{ij} \in \pi^*} p(e_{ij}) \right)^t \quad (3)$$

Algorithm 2: Edge Pruning Strategy

Input: $\mathcal{G} = (\mathcal{T}, E)$, $\{\mathcal{T}_Q^1, \mathcal{T}_Q^2, \dots, \mathcal{T}_Q^M\}$, t, f .
Output: $\mathcal{G}_s = (\mathcal{T}, E_s)$.

```

1  $E_s \leftarrow \emptyset$ ;  $\mathcal{E}_{paths} \leftarrow \emptyset$ ;
2 // Step 1: Candidate Path Selection
3 for  $k \in [1..M]$  do
4   for each pair  $(u, v) \in \mathcal{T}_Q^k \times \mathcal{T}_Q^k, u \neq v$  do
5      $\mathcal{P}_{\mathcal{G}}(u, v) \leftarrow \text{Find-TopK-Paths}(\mathcal{G}, u, v, f + 1)$ ;
6      $\mathcal{E}_{paths} \leftarrow \mathcal{E}_{paths} \cup \mathcal{P}_{\mathcal{G}}(u, v)$ ;
7 // Step 2: t-Spanner Pruning
8  $\mathcal{E}_{paths} \leftarrow \text{Sort-by-Weight}(\mathcal{E}_{paths})$ ;
9 for  $\pi_{uv} \in \mathcal{E}_{paths}$  do
10    $\text{count}_{\mathcal{G}_s}(u, v) \leftarrow \text{CountPaths}(u, v, \mathcal{T}, E_s)$ ;
11   if  $\text{ExceedStretch}(\mathcal{T}, E_s, \pi_{uv}, t)$  or  $\text{count}_{\mathcal{G}_s}(u, v) \leq f$  then
12      $E_s \leftarrow E_s \cup \text{Edges}(\pi_{uv})$ ;
13 return  $\mathcal{G}_s = (\mathcal{T}, E_s)$ ;

```

Intuitively, $\mathcal{G}_s$ is being constructed so that, for each important node pair (u, v) , the “good” connectivity of the original graph (i.e., paths not in F) is preserved inside the subgraph. That way, even if you remove up to f paths (e.g., the ones rejected by the LLM), the subgraph still correctly reflects the remaining valid paths necessary for answering the query.

Algorithm 2 outlines the process of deriving the subgraph $\mathcal{G}_s$. EcoTable first respectively initializes E_s and $\mathcal{E}_{paths}$ (the set of candidate paths) as empty (line 1). Then, in each iteration (lines 3–4), for each pair of nodes (i.e., tables) in $\mathcal{T}_Q^k$, we compute the top- $(f + 1)$ paths with the highest joinability in $\mathcal{G}$ (line 5) and add them to $\mathcal{E}_{paths}$ (line 6). Afterwards, we apply the t-Spanner algorithm to further prune edges in $\mathcal{E}_{paths}$. We sort the collected paths from highest to lowest joinability (line 8) and greedily add the edges of these paths to E_s . Concretely, the edges of the path with the highest joinability will be added. For each subsequent path π_{uv} in $\mathcal{E}_{paths}$ (line 9), its edges will be added if (1) the t -th power joinability of π_{uv} exceeds that of any of the top f paths in $\mathcal{G}_s$, which indicates that the bound w.r.t the t -th power of joinability is satisfied; or (2) there are fewer than f paths in $\mathcal{G}_s$ between u and v (the count is

computed by line 10), which indicates that the f -path fault-tolerant property is satisfied. Finally, the algorithm outputs the subgraph $\mathcal{G}_s$ (line 13).

Complexity of the pruning strategy. The computational cost of our pruning method consists of two stages. First, we compute the top- k join paths for every table pair in $\mathcal{T}_Q^k$. This requires $O(f \cdot M \cdot |\mathcal{T}_Q^k|^2 \cdot |E| \log |\mathcal{T}|)$, where M is the number of queries and $(f \cdot |E| \log |\mathcal{T}|)$ captures the cost of graph traversal (e.g., running Dijkstra) over $|\mathcal{T}|$ tables and $|E|$ edges. Second, we prune these candidate paths using t -spanner, which adds a cost of $O(f \cdot M \cdot |\mathcal{T}_Q^k|^2 \cdot |E_s| \log |\mathcal{T}|)$. Because the spanner is much sparser ($|E_s| \ll |E|$), the first stage dominates the overall runtime. In practice, EcoTable discards low-quality joins (e.g., $p(e_{ij}) < 0.3$) as a pre-processing step, ensuring $|E| \ll |\mathcal{T}|^2$, and limits path exploration to at most 5 hops to avoid path explosion in large graphs.

Complexity of graph-based validation. Eq. 2 reduces to the Minimum Steiner Tree problem, and pruning via Eq. 3 yields the Minimum t -Spanner problem; both are NP-hard [5, 27].

PROOF OF STEINER TREE REDUCTION. Considering a special case when $M = 1$, the problem then simplifies to $\max_{\{x_{ij}^1\}} \prod_{e_{ij} \in E} p(e_{ij})^{x_{ij}^1}$. Then, we can convert the problem to find a join path $\mathcal{G}_1 = (\mathcal{T}_1, E_1)$ containing all required tables $\mathcal{T}_Q^1$ from $\mathcal{G}$:

$$\begin{aligned} \max_{\{x_{ij}^1\}} \prod_{e_{ij} \in E} p(e_{ij})^{x_{ij}^1} &\iff \max_{\{x_{ij}^1\}} \ln \left(\prod_{e_{ij} \in E} p(e_{ij})^{x_{ij}^1} \right) \\ &\iff \min_{\{x_{ij}^1\}} \sum_{e_{ij} \in E} -\ln p(e_{ij}) x_{ij}^1 \end{aligned} \quad (4)$$

Next, we prove that the minimum Steiner tree problem [27] can be reduced to this special case in polynomial time. The minimum Steiner tree problem is defined as follows. Given a weighted graph $\mathcal{G}' = (\mathcal{T}', E')$ and a set of target nodes $R \subseteq \mathcal{T}'$, the goal is to determine a connected join path $\mathcal{G}_w = (\mathcal{T}_w, E_w) \subseteq \mathcal{G}'$ such that $R \subseteq \mathcal{T}_w$ has the minimum total weight $\sum_{e \in E_w} w'(e)$, where $w'(e)$ denotes the weight of edge $e \in E'$. Let $\mathcal{G}' = \mathcal{G}$, $\mathcal{T}' = \mathcal{T}$, $E' = E$, $w'(\cdot) = -\ln p(\cdot)$, and $\mathcal{T}_w = \mathcal{T}_Q^1$. Then, our problems share identical objectives and constraints with the classical minimum Steiner tree problem. Because the classical minimum Steiner tree problem is NP-hard, our problem is also NP-hard. $\square$

PROOF OF t -SPANNER REDUCTION. Considering a special case when $f = 0$ and the set of terminal nodes for all queries covers all possible pairs in the data lake, i.e., $\bigcup_{k=1}^M (\mathcal{T}_Q^k \times \mathcal{T}_Q^k) = \mathcal{T} \times \mathcal{T}$. The pruning problem according to Eq. 3 then simplifies to finding a subgraph $\mathcal{G}_s = (\mathcal{T}, E_s)$ from the global graph $\mathcal{G} = (\mathcal{T}, E)$ with minimum edges. For any table pair $(u, v) \in \mathcal{T} \times \mathcal{T}$, the maximum joinability path π_{uv} in $\mathcal{G}_s$ must satisfy:

$$\begin{aligned} \max_{\pi \in \mathcal{P}_{\mathcal{G}_s}(u,v)} \prod_{e_{ij} \in \pi} p(e_{ij}) &\geq \left(\max_{\pi^* \in \mathcal{P}_{\mathcal{G}}(u,v)} \prod_{e_{ij} \in \pi^*} p(e_{ij}) \right)^t \\ \iff \min_{\pi \in \mathcal{P}_{\mathcal{G}_s}(u,v)} \sum_{e_{ij} \in \pi} -\ln p(e_{ij}) &\leq t \cdot \min_{\pi^* \in \mathcal{P}_{\mathcal{G}}(u,v)} \sum_{e_{ij} \in \pi^*} -\ln p(e_{ij}) \end{aligned} \quad (5)$$

Next, we show that the classical minimum t -spanner problem reduces to this special case in polynomial time. The minimum t -spanner problem is defined as follows: given a weighted graph $\mathcal{G}' = (\mathcal{T}', E')$ and a stretch factor $t > 1$, find a subgraph $\mathcal{G}'_s = (\mathcal{T}', E'_s)$ with the minimum number of edges such that for every pair $(u, v) \in \mathcal{T}'$, the distance in $\mathcal{G}'_s$ is at most t times the distance in $\mathcal{G}'$. Let $\mathcal{T}' = \mathcal{T}$, $E' = E$, and $w'(\cdot) = -\ln p(\cdot)$. Then preserving t -th power joinability in the multiplicative space is equivalent to preserving a stretch factor t in the additive weight space. Since the minimum t -spanner problem is NP-hard for any $t > 1$, the edge pruning problem is also NP-hard. $\square$

Moreover, we analyze the Steiner Tree obtained in $\mathcal{G}_s$. We compute Steiner trees using a 2-approximation method (the KMB algorithm). Since $\mathcal{G}_s$ is a t -spanner of $\mathcal{G}$, this implies a $2t$ -approximation when working in $\mathcal{G}_s$. Consequently, the Steiner trees obtained in $\mathcal{G}_s$ achieves joinability within a factor of $2t$ of the optimal Steiner trees in $\mathcal{G}$.

We also bound the total number of LLM invocations N_{LLM} for M queries. The lower bound represents an ideal case where only edges truly belonging to the optimal Steiner trees are verified, with no extra or unnecessary checks. In contrast, the upper bound indicates the worst-case in which EcoTable might have to check every edge in the pruned subgraph $\mathcal{G}_s$. The number of edges $|E_s|$ provides this upper bound, as established by t -spanner theory. Therefore, we have:

$$\Omega \left(\left| \bigcup_{k=1}^M E_{OST}(\mathcal{T}_Q^k) \right| \right) \leq N_{LLM} \leq \tilde{O} \left(f \cdot |\mathcal{T}| \cdot \sum_{k=1}^M |\mathcal{T}_Q^k|^{1+\frac{2}{t+1}} \right) \quad (6)$$

where $E_{OST}(\mathcal{T}_Q^k)$ denotes the edge set of the optimal Steiner tree for query $\mathcal{Q}_k$.

5 TABLE TRANSFORMATION LAYER

The table transformation layer (1) automatically generates the transformation code that enables table joins (Sec. 5.1), and (2) executes these transformations efficiently through a graph-based parallel strategy (Sec. 5.2).

5.1 Transformation Execution

For each edge $e_k^{ij} \in E_k$, EcoTable uses a transformation module M_T to produce executable code to join the two columns in T_k^i and T_k^j specified by the LLM in the last later. Specifically, M_T identifies noise that interferes with equi-joins between the two tables and generates transformation code to fix these inconsistencies, thereby enabling an equi-join. As a modular component, M_T can be instantiated flexibly with various methods, such as DTT [8], TabulaX [44], or our proposed ReAct-style LLM agent. At a high level, the advantage of our approach is that it enables the agent to autonomously decide both when to invoke transformation operations and which specific ones are appropriate. Before proceeding, we must first give formal definitions of the grammars for several required operations.

Transformation Grammar. As shown in Table 1, we define 4 significant operators, organized into two functional categories:

Validate (O_{val}). Given the two columns corresponding to the tables, the *ValidateJoin* operator prompts the LLM to determine

Table 1: Transformation Grammar in EcoTable .

Category	Operator	Operator parameters
Validate	ValidateJoin	source_col, target_col
Transformation	Normalize [18]	input_col, target_format
	Split [56]	split_col, delimiter
	Pivot [55]	reshape_type, target_cols

whether a join between them is feasible. To support this decision, we supply the join containment ratio as a quantitative signal: $\text{Containment}(A, B) = \frac{|A \cap B|}{|A|}$. By comparing this ratio before and after each transformation, the agent checks whether the latest operation resolves inconsistencies. The procedure terminates and returns “joined” if the LLM concludes that no additional improvements can be made (for instance, when the ratio reaches 1), or when the maximum number of iterations is reached. Otherwise, the agent proceeds by applying further suitable operators from O_{trans} .

Transform (O_{trans}). We formally define the set of transformation operators used to support table joins as $O_{trans} = \{\text{Normalize}, \text{Split}, \text{Pivot}\}$. In particular, the Normalize operator allows the agent to generate distinct transformation functions for different formats within a single column (e.g., standardizing ISO strings and Unix timestamps together). The Split operator breaks a string column into two atomic columns based on a given delimiter (e.g., “_”), which facilitates accurate matching when dealing with composite keys. Lastly, the Pivot operator restructures the table so that schemas are aligned for downstream join operations.

Iterative ReAct Workflow. To summarize, the agent follows an iterative “Reason-Act-Observe” loop to join the tables. In each iteration: (1) The agent analyzes whether the tables need transformations based on the observation from previous iterations as well as current joinability score. If so, it selects the appropriate operators from the grammar. (2) The agent executes this code to apply specific transformations to the tables. (3) The environment captures the execution results and feedbacks the updated state. This loop persists until either the LLM determines as joinable or the maximum iteration limit is reached. Notably, this workflow supports multi-step transformation chains. For example, the agent can first invoke Pivot to align the table’s overall structure. It can then apply Normalize to resolve any remaining cell-level inconsistencies.

5.2 Parallelization Strategy

We obtain the validated Steiner trees $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_M\}$ in Sec. 4, where each $\mathcal{G}_k = (\mathcal{T}_k, E_k)$. These trees are merged to construct the composite graph $\mathcal{G}_v = (\mathcal{T}_v, E_v)$, where $\mathcal{T}_v = \bigcup_{k=1}^M \mathcal{T}_k$ and $E_v = \bigcup_{k=1}^M E_k$. A straightforward approach is to input the entire graph $\mathcal{G}_v$ to the ReAct-style agent and ask it to generate transformation code for every edge in that graph at once. However, this would yield suboptimal accuracy. The reason is that the tree structure of join relationships introduces complex sequential dependencies that make it difficult for LLMs to reason about all transformations coherently in a single pass. However, sequentially processing each edge would incur significant latency. To address this, we propose an iterative parallel processing strategy. In each iteration, we identify independent edges—those sharing no common nodes—within

Algorithm 3: Parallel Transformation Algorithm

Input: Validated composite graph $\mathcal{G}_v = (\mathcal{T}_v, E_v)$.
Output: Final integrated join path $\mathcal{G}^* = (\mathcal{T}^*, E^*)$.

```

1 // Step 1: Edge batching
2 BatchList  $\leftarrow$  Batch( $\mathcal{G}_v$ );
3  $\mathcal{T}^* \leftarrow \mathcal{T}_v$ ;
4  $E^* \leftarrow \emptyset$ ;
5 // Step 2: Execute batches in parallel
6 for each batch  $I_t \in \text{BatchList}$  do
7   for each edge  $e_{ij} \in I_t$  do in parallel
8      $(\hat{T}_i, \hat{T}_j, \hat{e}_{ij}) \leftarrow \text{ReAct\_Transform}(\mathcal{T}^*, e_{ij})$ ;
9      $\mathcal{T}^* \leftarrow (\mathcal{T}^* \setminus \{T_i, T_j\}) \cup \{\hat{T}_i, \hat{T}_j\}$ ;
10     $E^* \leftarrow E^* \cup \{\hat{e}_{ij}\}$ ;
11 return  $\mathcal{G}^* = (\mathcal{T}^*, E^*)$ ;

```

E_v and submit them to LLMs for parallel transformation. This approach ensures that transformations for independent edges can be reasoned about concurrently without cognitive interference. The process iterates until all edges in the join path have been processed, effectively balancing transformation quality with computational efficiency.

Maximizing Average Parallelism. We partition the edge set E_v into C disjoint batches $\{I_1, I_2, \dots, I_C\}$, where each batch $I_t, t \in [1, C]$ contains a set of independent edges that can be transformed in parallel. Then, our goal is to maximize the average parallelism of batches, which is defined as the mean number of transformations executed per batch, i.e., $\frac{1}{C} \sum_{t=1}^C |I_t|$.

Formally, the optimization objective could be formulated as:

$$\begin{aligned}
 \max_{\{I_t\}} \quad & \frac{1}{C} \sum_{t=1}^C |I_t| \\
 \text{s.t.} \quad & \bigcup_{t=1}^C I_t = E_v, \\
 & \forall t \neq t' : I_t \cap I_{t'} = \emptyset, \\
 & \forall e_{ij}, e_{uv} \in I_t : \{T_i, T_j\} \cap \{T_u, T_v\} = \emptyset.
 \end{aligned} \tag{7}$$

The second constraint ensures that the edges do not overlap between batches. The third constraint ensures that no two edges in one batch share the same table.

Parallel Transformation Algorithm. In Equation 7, our goal is to choose as many independent edges as possible per batch, which corresponds to minimizing the total number of batches. This can be regarded as the classical edge-coloring problem [26, 57], which assigns colors to edges such that no two adjacent edges share the same color, equivalent to our problem of finding independent edges. The goal is to minimize the number of colors used, which corresponds to the number of batches in our setting. EcoTable uses the typical edge coloring algorithm, i.e., Vizing [57] to identify a near-optimal list of batches (line 2, as shown in Algorithm 3). For each batch I_t , EcoTable executes all transformations in parallel using the ReAct reasoning loop, updates transformed tables, and records validated edges (line 3 - 10). In the end, all transformed tables and validated edges form the final integrated join path (line 11).

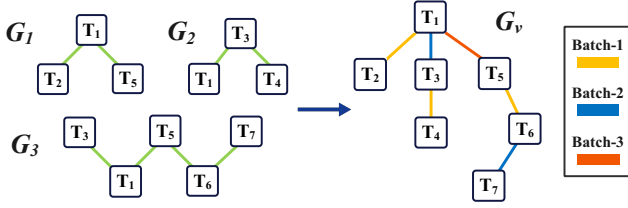


Figure 4: Parallelization Strategy

Next, we provide an example to demonstrate how EcoTable organizes transformations across multiple queries into parallel batches.

Example 5.1. As shown in Figure 4, we merge the Steiner tree $\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3$ into the composite graph $\mathcal{G}_v$. We then apply the Vizing algorithm to color the edges of $\mathcal{G}_v$, which produces three batches of independent edges, i.e., Batch-1 = $\{e_{12}, e_{34}, e_{56}\}$, Batch-2 = $\{e_{13}, e_{67}\}$, and Batch-3 = $\{e_{15}\}$. Each batch runs simultaneously.

6 Evaluation

In this section, we first detail our benchmark construction process (Sec. 6.1), followed by extensive experiments to compare EcoTable against 6 baselines in both effectiveness and efficiency, focusing on answering the following research questions (RQs):

RQ1 (Sec. 6.2): How does EcoTable perform in discovering the correct join paths compared to baseline approaches?

RQ2 (Sec. 6.3): Can EcoTable improve the accuracy of end-to-end SQL execution when combined with an Text-to-SQL system?

RQ3 (Sec. 6.4): How do the execution time and token cost of EcoTable scale as the total number of tables in the data lake increases?

RQ4 (Sec. 6.5): How effective is the two-stage architecture in the table identification layer at retrieving relevant tables, and what is the impact of different PLM backbones?

RQ5 (Sec. 6.6): How does the graph-based validation layer perform in estimating joinability and balancing accuracy with cost?

RQ6 (Sec. 6.7): How effective is the ReAct-style method in generating accurate table transformations, and how does the graph-based parallelization strategy improve execution efficiency?

RQ7 (Sec. 6.8): What are the main LLM error types in join validation and transformation generation, and what causes these failures in complex data lake settings?

RQ8 (Sec. 6.9): How do key hyperparameters (e.g., candidate table and training data ratios) impact EcoTable’s performance?

6.1 Experimental Settings

Datasets. We evaluate the performance of EcoTable across five benchmark datasets, categorized two groups: four real-world industrial benchmarks and a large-scale synthetic data lake. The statistics of these datasets are summarized in Table 2.

(i) **DBT Benchmarks.** We derive four datasets from well-known DBT datasets for table integration and transformation [9]. Dataset statistics are shown in Table 2. Specifically, (1) Ad represents advertising analytics scenarios and includes 69 tables and 126 natural language queries. Each query typically involves information such as spend, impressions and clicks, with an average of 3.65 joins per query. (2) Engagement captures user engagement and communication data, including email campaigns, in-product events,

and support ticket records. It contains 56 tables and 38 queries, averaging 2.76 joins per query. (3) Business consists of 90 tables and 28 queries. It integrates financial ledgers, orders, and sales management data, averaging 4.33 joins per query. (4) Platform captures engineering and platform activities, such as code repositories, project management, and data pipeline logs. It includes 35 tables and 24 queries, averaging 2.67 joins per query. Our benchmark features columns ranging from 2 to 389 for each table, with row counts varying significantly from 1 to 4,033, highlighting the complexity of integration.

In Figure 5 (left), each DBT dataset includes a set of interrelated tables and SQL statements that serve as the basis for our ground truth generation and query construction. The datasets offer original raw tables in the data lake (green tables T_1, T_2, T_3), transformed tables derived from these raw tables ($\hat{T}_1, \hat{T}_2, \hat{T}_3$), and integrated tables T^* , created by joining transformed tables (e.g., T^* from $\hat{T}_1, \hat{T}_2, \hat{T}_3$). Transformations and joins are defined by SQL statements, detailing how tables like $\hat{T}_1, \hat{T}_2, \hat{T}_3$ transform from T_1, T_2, T_3 and how they join to form T^* .

To construct our benchmark, overall, we start by translating the provided SQL statements to the ground-truth join paths. We then automatically generate the corresponding natural language queries that semantically align with these join paths.

Ground Truth Exaction. As shown in Figure 5 (middle), for each join SQL statement that corresponds to an integrated table T^* , we use LLMs to parse the SQL statement, identify the participating tables and join conditions, and then transform it into a ground-truth join path $\mathcal{G}^* = (\mathcal{T}^*, E^*)$, where $\mathcal{T}^* = \{\hat{T}_1, \hat{T}_2, \hat{T}_3\}$ and E^* denote valid join relationships inferred from the SQL join conditions. To ensure correctness, we ask human experts to check and correct the join paths.

Query Construction. According to the ground-truth join path, we design two categories of queries based on whether bridge tables are required during integration:

(1) **Queries Requiring Bridge Tables:** For this type of query, we begin by randomly selecting multiple subsets of non-leaf nodes from the ground-truth join path to serve as bridge tables. Once the bridge tables are selected (i.e., $\hat{T}_1 \in \mathcal{T}^*$ in Q_2 from Figure 5 (right)), they are removed from the join path, and the remaining tables ($\hat{T}_2, \hat{T}_3 \in \mathcal{T}^*$) are examined to identify relevant columns that will serve as attributes in the user’s query. These selected columns are then fed to LLMs to construct the corresponding natural language query.

(2) **Queries Not Requiring Bridge Tables:** Since this type of queries requires columns from all tables in $\mathcal{T}^*$ involved in the ground truth join path, we can simply translate the join SQL statement into a natural language query using LLMs.

Overall, 67.13% of queries require bridge tables (145/216), whereas 32.87% do not (71/216).

(ii) **NYC Data Lake.** To further test the scalability of EcoTable and its robustness, we introduce a large-scale benchmark constructed from NYC Open Data [7]. Unlike the DBT benchmarks, this dataset is generated to simulate a massive, messy data lake environment. It comprises 1,214 tables and 800 queries, featuring different types of noise and a large search space designed to challenge table identification and join path discovery algorithms. Next, we illustrate

Table 2: Statistics of Datasets.

Dataset	#-Tables	#-Queries	#-Max/Min/Avg. Column	#-Max/Min/Avg. Row	Avg. #-Tables/Join Relations Per Query
Ad [13]	69	126	86 / 3 / 19.51	2630 / 1 / 99.51	4.65 / 3.65
Engagement [15]	56	38	48 / 3 / 14.11	100 / 1 / 20.32	3.76 / 2.76
Business [14]	90	28	389 / 2 / 48.66	4033 / 1 / 100.48	5.33 / 4.33
Platform [17]	35	24	15 / 3 / 7.43	200 / 1 / 38.29	3.67 / 2.67
NYC Data Lake [16]	1214	800	219 / 1 / 19.67	480 / 1 / 183.58	5.04 / 4.14

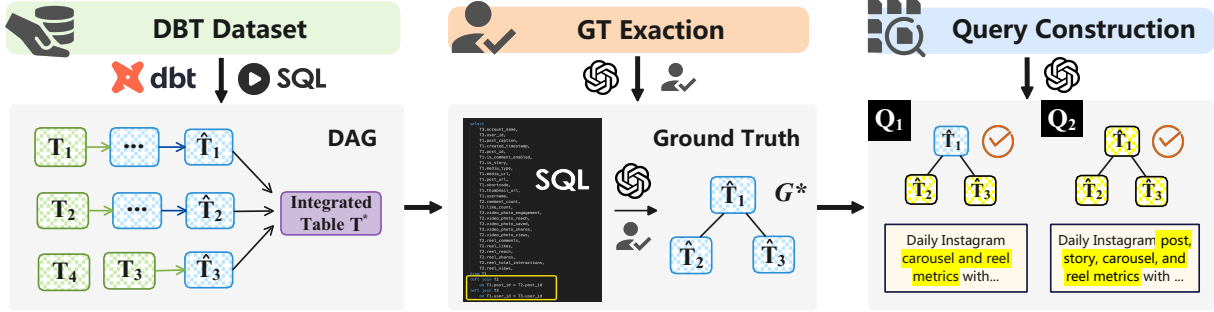


Figure 5: DBT Benchmark Datasets Construction.

the concrete process of table generation, join path generation and query construction.

Real-World Data Lake (Tables & Join Paths) Construction. At a high level, besides the larger size, the constructed data lake should have two key characteristics: (1) it should contain join ambiguity and schema noise to reflect the messy real-world scenario; and (2) it should include a number of tables that support multi-table join to answer our queries.

To this end, we first select a set of 100 tables with the largest number of columns from the original large NYC Open Data. Then, for each table T , we retrieve top- k ($k=3$) tables from the original Data Lake that are the most joinable with T , which is measured by DeepJoin [12], but the three tables might not successfully join with T . This step introduces both join ambiguity and schema noise because the NYC Open Data naturally contains noisy information like incomplete values, unaligned schemes, fuzzy joins, etc.

Second, we split each table with large number of columns to multiple sub-tables to form multi-join relationships. We first identify high-cardinality columns (many distinct values) and designate them as join keys, which connect nodes along the join path. All other columns are treated as non-join-key attributes and partitioned into disjoint subsets, each assigned to a different sub-table (node) on the path. The number and layout of sub-tables follow the join-path template defined in the DBT project. Adjacent nodes share at least one high-cardinality join-key column, which defines their join condition.

To better reflect realistic settings, we conduct the following steps over these sub-tables. (1) Since real-world multi-table joins often produce sparse matches, to simulate this, we restrict the overlap to only 5% of original tables and assign the remaining 95% of rows into different sub-tables. (2) We inject cell-level (e.g., case changes, whitespace removal, typos, or number variations) noise to 37% join columns. (3) We ask humans to identify 8% columns to merge/split

to produce column-level noise. (4) We also apply pivot/unpivot operations over 4.8% columns. Overall, we construct over 521 joinable tables with explicit join-path ground truth.

Third, for all the tables considered above (denoted as the set $\mathcal{T}$), the most direct way to obtain ground truth would be to have humans examine every possible table pair to determine joinability, which is prohibitively costly. To mitigate this, for each table T and each $T' \in \mathcal{T}$, we only send the pair to human evaluators if $\text{DeepJoin}(T, T') > 0.8$; otherwise, we treat the pair as non-joinable. Because some join paths have already been identified as ground truth in the second step above, the amount of required human annotation is substantially reduced.

Query Construction. In this new benchmark, we create a total of 800 queries, consisting of 100 human-written queries and 700 LLM-generated queries. For the 100 human-written queries, we strictly follow the annotation pipeline described in the “Incorporation of human-written queries” part above. For the 700 LLM-generated queries, we use the same construction process as for the four original datasets. Specifically, this process involves three steps: (1) For each join path in the data lake, we first prompt LLMs to instantiate predefined SQL templates, thereby producing candidate SQL statements. (2) We then execute these SQL statements to validate their effectiveness, directly discarding any that returns excessive NULL values or duplicate rows. (3) Finally, we ask LLMs to convert the remaining valid SQLs into natural language questions. Human annotators subsequently revise these questions by hand to ensure natural fluency while strictly maintaining equivalence with the underlying SQL.

Baselines. We compare EcoTable with various baselines, including existing methods and variants of our own approach.

(1) Auto-BI [40] identifies a join path using deep learning models given multiple tables. For fair comparison, we apply our table identification layer to capture query-relevant tables. Auto-BI then optimizes the join path by targeting these relevant tables.

(2) Auto-Prep [29] jointly predicts transformations and join paths with dual deep-learning models. It also considers the entire data lake tables rather than focusing on tables specific to an NL query. To ensure a fair comparison, we utilize our table identification layer to identify the tables relevant to the query. Subsequently, Auto-Prep performs searches for transformations and joins within these relevant tables.

(3) LLMs-Only-EcoTable (LOE) relies exclusively on LLMs for the entire pipeline, performing table identification, join path discovery, and transformation code generation without graph-based optimization.

(4) Search-with-Greedy-Algorithm (SGA) is a variation of our method that incrementally selects join edges with the highest joinability scores until all relevant tables are connected.

(5) Without-ReAct-Reasoning (WRR) is another variation that removes the ReAct-style loop, utilizing LLMs to produce the final transformation code directly in a single step.

(6) EcoTable is our full-fledged solution. It uses DeBERTa-v3-Large for M_S , DeepJoin for M_J , and the ReAct-style agent for M_T .

Metrics. We evaluate the join path accuracy, success rate for Text-to-SQL, average cost and average execution time across all the datasets. (1) Join Path Accuracy. We take F1 score as the evaluation metric. Let $\mathcal{G}_k^{gt} = (\mathcal{T}_k^{gt}, E_k^{gt})$ and $\mathcal{G}_k^{pred} = (\mathcal{T}_k^{pred}, E_k^{pred})$ denote the ground-truth and predicted join paths for query Q_k , respectively. We employ LLMs to judge whether the two edges represent the same join relation. Then, the precision and recall are computed as $\text{Pre}_k = \frac{|E_k^{pred} \cap E_k^{gt}|}{|E_k^{pred}|}$ and $\text{Rec}_k = \frac{|E_k^{pred} \cap E_k^{gt}|}{|E_k^{gt}|}$ respectively. The F1

score for Q_k is $F1_k = \frac{2 \times \text{Pre}_k \times \text{Rec}_k}{\text{Pre}_k + \text{Rec}_k}$. The overall accuracy is $\overline{F1} = \frac{1}{M} \sum_{k=1}^M F1_k$, where M is the total number of queries. (2) Success Rate for Text-to-SQL. We follow [32] to adopt the success rate (SR) metric, which is commonly used in Text-to-SQL evaluation [32] to measure the fraction of queries that are correctly answered. (3) Transformation F1-score. Following [8], we evaluate transformation quality by comparing the system-generated join cell pairs with the ground-truth pairs. (4) Average Cost. For each query Q_k , let C_k represent the total expense incurred by all LLM calls, which is calculated based on the API pricing [1, 45] calculated considering token usage. The average cost is determined as $\overline{C} = \frac{1}{M} \sum_{k=1}^M C_k$. (5) Cost-Aware Utility score (CAU). This score combines the F1 score and the normalized cost efficiency using a weight factor α to balance their importance: $\text{CAU} = \alpha \cdot F1 + (1 - \alpha) \cdot \left(1 - \frac{\text{Cost} - \text{Cost}_{\min}}{\text{Cost}_{\max} - \text{Cost}_{\min}}\right)$. We set $\alpha = 0.8$ in this paper. (6) Average Execution Time. We evaluate the average execution time as $\overline{t} = \frac{1}{M} \sum_{k=1}^M t_k$, where t_k denotes the end-to-end execution time.

6.2 Comparison with Baselines (RQ1)

6.2.1 Join Path Accuracy w.r.t. All Queries. Figure 6 reports the average F1 scores in join path accuracy with baselines. These methods are ranked as follows: EcoTable $\approx$ LOE $\approx$ SGA $>$ WRR $>$ Auto-Prep $>$ Auto-BI.

EcoTable and LOE achieve the highest F1 scores among all baselines. LOE performs well because it leverages the powerful reasoning capabilities of LLMs to perform end-to-end join path discovery. EcoTable is competitive to LOE but incurs significantly lower costs.

This is achieved by integrating a lightweight deep learning model with a Steiner tree formulation to identify a highly probable join path, which is then validated by LLMs solely on this path, thus reducing unnecessary LLMs calls while preserving high quality. Empirically, SGA and EcoTable achieve similar F1 because SGA also uses LLMs to validate edges but incurs higher costs because of the suboptimal edge selection strategy. Compared to WRR, EcoTable attains higher accuracy. because the ReAct mechanism effectively enables LLMs to detect and fix local reasoning errors during the transformation generation process.

EcoTable performs better than Auto-BI, for instance, on Ad, EcoTable achieves an F1 score of 79.2%, significantly outperforming Auto-BI (21.5%). This is due to (1) Auto-BI failing to consider table transformations, leading to a failure in joining two tables, and (2) Auto-BI depending solely on a deep model to estimate pairwise joinability. In contrast, EcoTable incorporates both table transformations and join path discovery using LLMs.

Notably, on NYC dataset, EcoTable and SGA outperform LOE, which is different from the observations in other benchmarks. The F1-score of LOE drops to 52% because its initial schema linking step introduces many query-irrelevant tables that significantly reduce precision. In contrast, EcoTable maintains a high F1 score of 65% by using a deep learning model to filter irrelevant tables, which ensures high recall while preventing LLM performance degradation due the large number of noise tables. Consequently, EcoTable proves more robust than end-to-end LLM methods when dealing with large-scale data lakes containing noisy data.

6.2.2 Average Cost. Figure 7 reports the average cost compared with baselines. EcoTable demonstrates a significant advantage in cost-efficiency compared to all baseline methods.

EcoTable has substantially lower cost than LOE. On average for all datasets, EcoTable reduces token cost by about 5.3 $\times$ compared to the greedy baseline SGA. This is because EcoTable leverages the Steiner-tree to achieve a global optimal edge selection, i.e., always identifying the most likely join paths for user-specified queries to validate. EcoTable also remains cheaper than LOE, which relies on end-to-end LLM exploration of the entire join space. Auto-BI and Auto-Prep use fewer costs because they only use LLMs for relevant table identification. However, they are not accurate enough, as illustrated in Sec. 6.2.1. EcoTable is slightly more expensive than WRR because the ReAct reasoning loop in EcoTable involves multiple iterative LLMs calls per transformation. However, the transformation code generated in a single LLM call often produces erroneous code due to a lack of execution feedback and step-by-step reasoning, resulting in a low F1 score of WRR. On the NYC dataset, the cost of EcoTable rises because the join validation algorithm may exhaustively traverse edges to find a valid path in a large search space. To improve scalability, Section 6.4 presents a variant of EcoTable with a sophisticated edge pruning strategy tailored for massive data lakes.

6.3 Integration with Text-to-SQL Systems (RQ2)

We evaluate how our method affects the SQL execution accuracy when integrated with a Text-to-SQL model. We integrate our method

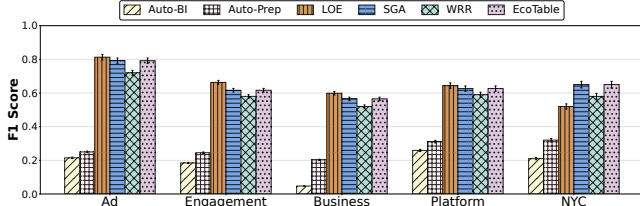


Figure 6: Comparison of Join Path Accuracy (F1 Score).

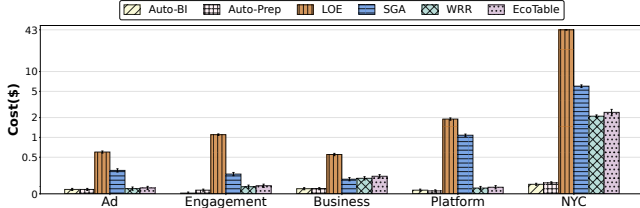


Figure 7: Comparison of Token Cost per Query.

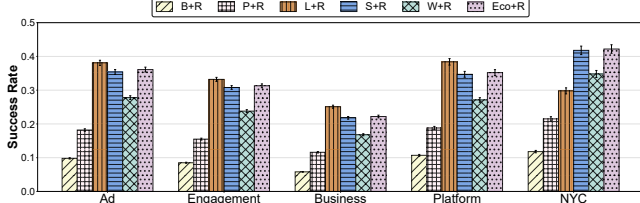


Figure 8: End-to-end SQL Success Rate.

with variant Text-to-SQL methods, including ReFoRCE [10], MAC-SQL [58], RSL-SQL [4] and Spider-Agent [32]. Due to space limitation, we only illustrate the results on the execution accuracy when integrated with ReFoRCE. Furthermore, we compare EcoTable with the following baselines:

- (1) Auto-BI + ReFoRCE (B+R) combines the join paths identified by Auto-BI with ReFoRCE.
- (2) Auto-Prep + ReFoRCE (P+R) merges the join paths detected by Auto-Prep with ReFoRCE.
- (3) LOE + ReFoRCE (L+R) integrates the join paths found by LOE with ReFoRCE.
- (4) SGA + ReFoRCE (S+R) incorporates the join paths generated by SGA with ReFoRCE.
- (5) WRR + ReFoRCE (W+R) leverages the join paths produced by WRR with ReFoRCE.
- (6) EcoTable + ReFoRCE (Eco+R) uses EcoTable to obtain verified join paths for ReFoRCE.

In Figure 8, our approach Eco+R achieves a competitive success rate compared to S+R and L+R across most datasets, but with significantly lower costs as illustrated in Sec. 6.2.2. Eco+R outperforms both B+R and P+R because both methods adopt a deep learning model to estimate pairwise joinability without LLMs validation, thus introducing incorrect join relationships to the Text-to-SQL model. Eco+R outperforms W+R because its ReAct-based transformations effectively resolve data inconsistencies, providing clean tables that enable the Text-to-SQL model to generate executable and correct answers for the query. Empirically, L+R and Eco+R achieve similar success rates on DBT benchmarks. However, on the NYC dataset, the performance of L+R drops because it introduces many

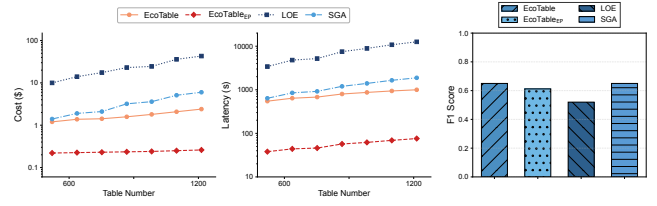


Figure 9: Scalability Evaluation.

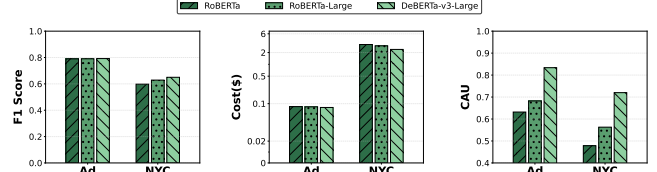


Figure 10: Impact of PLM Backbones.

query-irrelevant tables, which confuses the Text-to-SQL model and leads to incorrect join logic in the generated SQL.

6.4 Scalability Evaluation. (RQ3)

We evaluate how both the end-to-end runtime and token cost scale with the total number of tables in NYC Data Lake. Specifically, given the 800 queries, besides the tables necessary for answering these queries, we additionally add other tables into the data lake to incrementally expand the data lake size. The x-axis represents the number of added tables. Each query execution is repeated 10 times at each data scale.

As illustrated in Figure 9 (a) and (b), we can observe that with the number of table increasing, EcoTable spends much more time and LLM costs than EcoTable_{EP} because EcoTable_{EP} prunes the original graph to a compact subgraph, and thus the search space is much reduced. EcoTable_{EP} is also more efficient than LOE and SGA, which further demonstrates its superior performance. Besides, we can observe that in Figure 9 (c), EcoTable_{EP} achieves a similar performance with EcoTable, which does not sacrifice the accuracy because our pruned graph theoretically preserves the joinability property of the original large graph.

6.5 Eval. of Table Identification Layer (RQ4)

We evaluate the table identification layer in EcoTable by focusing on two aspects: the impact of the underlying Pre-trained Language Model (PLM) and the effectiveness of our two-stage architecture.

6.5.1 Impact of PLM Backbones. We compare multiple state-of-the-art models to evaluate how various PLMs influence the result. In particular, we evaluate RoBERTa, RoBERTa-Large, and DeBERTa-v3-Large. As shown in Figure 10, These three models achieve similar accuracy and cost although DeBERTa-v3-Large uses a larger model, because we subsequently leverage the LLM to verify the results, which demonstrates that EcoTable is robust on the model used for schema linking.

6.5.2 Effectiveness of Table Identification Layer. We further evaluate the effectiveness of EcoTable’s table identification layer by comparing it against 3 baselines:

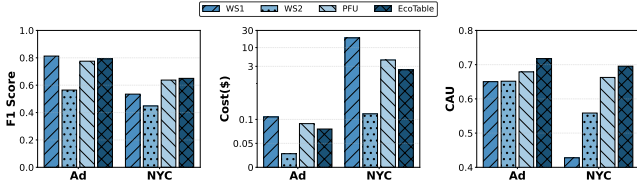


Figure 11: Evaluation of Table Identification Layer.

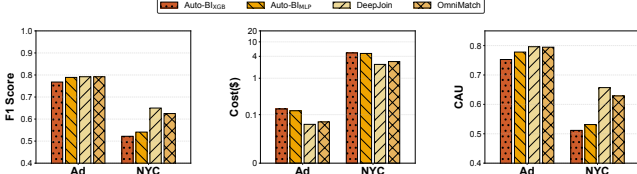


Figure 12: Evaluation of Graph-based Validation Layer.

- (1) Without-Stage-1 (WS1) removes the PLM-based coarse filtering in stage 1 from EcoTable and directly uses LLMs for schema linking.
- (2) Without-Stage-2 (WS2) removes the LLMs refinement in stage 2 from EcoTable and keeps only the deep-model filtering.
- (3) Post-Filtering-Union (PFU) uses LLMs to decompose a complex query into sub-queries, applies PLM-based filtering for each sub-query to find relevant tables, and then unions these sets before the LLM verification.

Figure 11 illustrates the Join Path F1-score for all methods. EcoTable achieves the best performance across all baselines. EcoTable outperforms WS1 because the deep-model filters irrelevant tables, ensuring high recall while preventing unnecessary LLM calls. Compared with WS2, EcoTable improves the average recall by 8% across all the datasets with only a marginal increase in LLM invocations, as the LLM verification eliminates false positives from the deep-model filtering. EcoTable and PFU are competitive in Join Path F1 (about 61%), but our method is cost-effective, because our method adopts a conservative filter strategy (by setting k to a larger value, say 30) to discard tables that are unlikely to be relevant to a query.

6.6 Eval. of Graph-based Validation Layer (RQ5)

We conduct a comprehensive evaluation of the Graph-based Validation Layer to validate the effectiveness of our joinability estimation module. Specifically, we compare EcoTable against the following methods:

- (1) Auto-BI_{MLP} [40] takes a pair of columns, encodes them with 24 handcrafted features and feeds them into a four-layer MLP to produce a probability.
- (2) Auto-BI_{XGB} [40] relies on the same set of 24 manually engineered features, but performs classification using a Gradient Boosting Decision Tree estimators.
- (3) OmniMatch [28] encodes each table column using statistical features and contextual embeddings learned with a Relational Graph Convolutional Network, which aggregates information from similar columns to refine their vectors and then computes distances between these vectors as a joinability probability.
- (4) DeepJoin [12] converts table columns into textual sequences. It then applies a pre-trained MPNet model to obtain dense vector

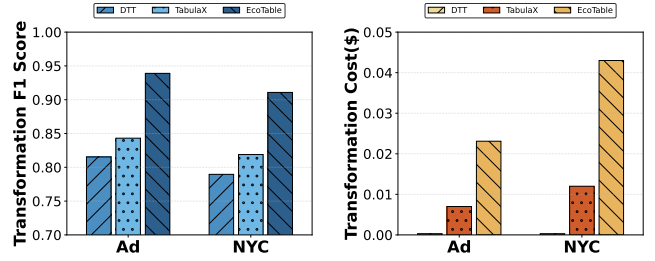


Figure 13: Evaluation of Transformation Generation.

embeddings and assesses joinability by computing the cosine similarity between them.

We can observe from Figure 12 that DeepJoin and OmniMatch achieve a higher accuracy than Auto-BI because the it leverages the pre-trained language model to compute the joinability, while Auto-BI just uses the hand-crafted features. Auto-BI also costs more because its low accuracy causes the LLM to check more edges.

6.7 Eval. of Table Transformation Layer (RQ6)

6.7.1 Effectiveness of Transformation Generation. To evaluate the accuracy of our ReAct-style method, we conduct a standalone evaluation comparing it against two automated baselines, i.e., DTT [8] and TabulaX [44] using the Transformation F1-score.

- DTT [8]: A PLM-based baseline for table transformation. Given pairs of target columns that need to be aligned, DTT uses the original cell contents as input and directly produces the desired target values. The source columns are subsequently revised by mapping their cell values to these generated outputs, altering the tables so they satisfy equality-based join conditions.
- TabulaX [44]: An LLM-driven method for multi-class table transformations. The baseline first classifies each input column pair into one of four types: string-based, numerical, algorithmic, or general. Based on this, TabulaX uses LLMs to derive transformation rules, which are then applied to convert original cell values into target formats, enabling equality-based joins.

As shown in Figure 13, EcoTable achieves a higher Transformation F1-score than both baselines because the ReAct-style reasoning dynamically composes atomic operators to resolve complex inconsistencies, whereas one-shot approaches like DTT and TabulaX struggle with unpredictable structural noise.

6.7.2 Efficiency of Parallelization Strategy. To evaluate the computational efficiency of our graph-based parallelization, we compare EcoTable against two execution baselines:

- Sequential-Execution(SE) executes transformations sequentially. For the graph $\mathcal{G}_v$, it sequentially invokes the ReAct-style agent on each edge to transform the table pairs and update the graph.
- Composite-Graph (CG) feeds the complete graph $\mathcal{G}_v$ to the ReAct-style agent and requests it to produce transformation code for all edges in the graph simultaneously.

In Figure 14, our method achieves an average execution time of 6.29 seconds per query without compromising accuracy, making it 3.5 times more efficient than SE. This is because our parallel scheduling mechanism enables the concurrent execution of independent

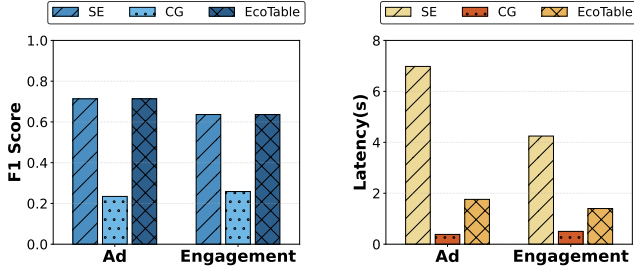


Figure 14: Evaluation of Parallel Execution.

Table 3: The Percentage of Different LLM Error Modes (%).

Task	Error Mode	Model		
		GPT-4o	Qwen3Max	DeepSeek3.2
Join Validation	Semantics	68	53	45
	Domain Knowledge	32	47	55
Transformation.	Semantics	55	70	67
	Domain Knowledge	32	21	25
	Syntax	13	9	8

join validations and transformations, thereby optimizing resource utilization. Although CG is highly efficient, it exhibits lower join path accuracy than EcoTable because the complex dependencies among transformations across the join path lead to hallucinations in LLMs or incorrect code for a significant portion of the edges.

6.8 Error Analysis (RQ7)

To understand where and why the LLM fails, we manually reviewed 100 error cases and categorized the LLM error modes in EcoTable into the following three types:

- (1) *Semantics Errors*. The LLM fails to understand the underlying semantic meaning of the data due to the contextual ambiguity.
- (2) *Domain Knowledge Errors*. The LLM lacks external, domain-specific knowledge to reason correctly.
- (3) *Syntax Errors*. The LLM generates transformation code that fails at runtime due to syntax mistakes.

As shown in Table 3, we compare the error distributions w.r.t. GPT-4o, Qwen3Max, and Deepseek3.2. These three types of errors manifest differently in our two core tasks: in join validation, the focus is mainly on textual reasoning instead of code execution, so syntax errors are irrelevant. Thus, only the other two error modes are observed in this task.

- (1) *Semantics Errors*: for instance, consider an Internal_Part_Number in a factory table (i.e., an ID used within a company’s manufacturing or inventory system) and a Supplier_Part_Number in a vendor table (i.e., an ID used by an external supplier in their catalog). Since the column names are similar and share example values (e.g., “PN-8832”), the LLM may treat them as directly joinable. In fact, they are IDs from separate, unrelated systems.
- (2) *Domain Knowledge Errors*: for example, a data lake includes a numeric Code column (e.g., “541511”) and a textual Course_Name column (e.g., “Computer Programming”) that actually refer to the same entity. However, the LLM may still reject this relationship, because it lacks the domain knowledge that “541511” corresponds to “Computer Programming”.

In Transformation Generation, we observe all the three error modes:

- (1) *Semantics Errors*: for example, given Company_Name column (e.g., “Alphabet Inc”) and Vendor column (e.g., “Google Cloud LLC”), the LLM relies on its prior world knowledge to equate the subsidiary with the parent company. It uses a Normalize operator to map “Google Cloud LLC” directly to “Alphabet Inc.”. However, this transformation is invalid because these are two distinct legal entities in corporate tax filings, leading to a logically incorrect aggregation of financial records.
- (2) *Domain Knowledge Errors*: for instance, Table A contains a 10-digit BBL_Code (e.g., “1000477501”), while Table B has separate Borough, Block, and Lot columns. After splitting the 10-digit code, the LLM must map the first digit (“1”) to “Manhattan”. Without possessing this city-specific knowledge, the LLM cannot generate the correct transformation code to align the two tables.
- (3) *Syntax Errors*: for instance, consider a multi-step transformation where the LLM first performs a Split operator on a Full_Address column. The execution engine automatically assigns default names to the resulting output columns (e.g., Full_Address_part_1). However, in the subsequent step, the generated code mistakenly refers to a semantic column name that does not exist (e.g., Address) when applying the Normalize operator. This inconsistency between the column name used in the generated code and the actual intermediate table columns results in a runtime error.

We can observe from the above table that the semantic errors occur the most frequently, mainly because (1) the number of ambiguous values in data lake columns is naturally large; (2) LLMs tend to conduct over-reasoning from general language patterns; and (3) database joins require exact logical matches, but LLMs operate on probabilistic language patterns that favor broad similarity over strict logic. Domain knowledge errors happen less because LLMs have already memorized extensive knowledge during pre-training, so it is less frequent than the semantic errors. In addition, the number of syntax errors is the smallest because we have an explicit grammar definition and a ReAct-style reasoning process.

6.9 Ablation Studies (RQ8)

6.9.1 Ratio of Candidate Tables in Stage 1. We evaluate the impact of the ratio of candidate tables $\frac{\tau_{cand}^k}{\tau}$. As shown in Figure 15, we observe that a small ratio misses relevant tables, causing incomplete join paths and lower join path accuracy. While too large ratio leads to diminishing join path accuracy gains but significant cost overhead. Based on our empirical observation, setting $\frac{\tau_{cand}^k}{\tau} = 0.3$ achieves the best balance between effectiveness and costs.

6.9.2 Training Data Ratio. We also evaluate how the ratio of training data used to fine-tune the deep model (i.e., RoBERTa [41]) affects the performance of EcoTable. As shown in Figure 16, with the ratio increases, the average F1 score improves until the ratio reaches 0.6. Therefore, a ratio around 0.6 appears to be suitable based on empirical observations.

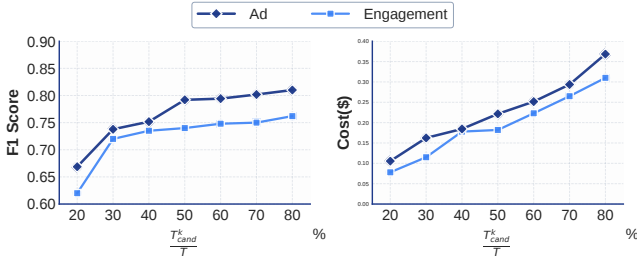


Figure 15: Effect of the Ratio of Candidate Tables.

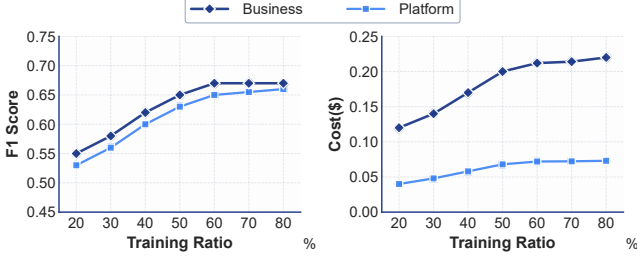


Figure 16: Effect of the Ratio of Training Samples.

7 RELATED WORKS

Text-to-SQL. Modern Text-to-SQL methods primarily follow two paradigms: fine-tuning, which adapts models to specific database schemas [34, 35, 50] and LLM-based prompting, which leverages in-context reasoning without parameter updates [30, 48, 66]. As more challenging benchmarks such as Spider 2.0 [32] and BIRD [36] emerge, recent work increasingly focuses on strengthening reasoning. For instance, RSL-SQL [4] targets the semantic gap through schema linking, while MAC-SQL [58] and ReFoRCE [10] employ multi-agent interaction and iterative refinement to address ambiguous queries. Despite these advances, existing approaches assume that the databases that perfectly support user queries exist a priori, while our work targets lightly structured data lakes where no database schema is available beforehand.

Joinable Table Discovery. Discovering joinable tables in data lakes is essential for data integration. Its key component is to compute the joinability between two tables, which is also a significant step in our graph-based validation layer. To be specific, early approaches such as Josie [68] and LSH Ensemble [70] rely on set-level overlap between column values to measure the joinability. Pexeso [11] and DeepJoin [12] leverage semantic embeddings from pre-trained language models to capture fuzzy joinability. OmniMatch [28] encodes each table column using statistical features and contextual embeddings learned with a Relational Graph Convolutional Network, which aggregates information from similar columns to refine their vectors and then computes distances between these vectors as a joinability probability. Besides, schema matching methods [2, 42, 47, 52] could also be utilized to compute the joinability based on the matching scores between two columns. In contrast to these methods, which use a table to identify joinable tables in a data lake, we focus on processing natural language queries to integrate multiple tables within a data lake.

Table Transformation. Table transformation methods fall into two main categories: traditional methods [8, 63, 69] and LLM-based methods [38, 44]. For the former, early approaches like AutoJoin [69] and Auto-Pipeline [64] rely on predefined operators and structural metadata to automate join discovery and pipeline construction, but are constrained by fixed operator sets. DTT [8] instead fine-tunes the PLM in an example-driven way to reformat data values. Among LLM-based methods, TableGPT [38] improves performance on table-related tasks via instruction tuning, while TabulaX [44] classifies columns into four transformation types and applies chain-of-thought reasoning to generate interpretable functions. Unlike prior methods, we directly transform two given tables into a unified format that directly supports equi-joins for the specific join needs of each query.

8 Conclusion

We introduced EcoTable, a cost-effective framework for integrating data lake tables via natural language queries. EcoTable uses a graph-based framework, combining lightweight models with LLMs to efficiently narrow the search space and minimize costly LLM calls through prioritized validation. Experiments show EcoTable achieves high integration accuracy but with lower costs.

Limitation. Our main focus is on queries with acyclic join graphs. We recognize that some complex queries might involve cyclic join graphs. Also, there might be multiple join relationships between a table pair. These are valuable issues that we plan to explore in future work.

References

- [1] Anthropic. n.d.. Anthropic API Pricing. <https://www.anthropic.com/pricing>. [n.d.] Website, accessed Nov. 1, 2025.
- [2] David Aumüller, Hong Hai Do, Sabine Massmann, and Erhard Rahm. 2005. Schema and ontology matching with COMA++. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, Baltimore, Maryland, USA, June 14-16, 2005, Fatma Özcan (Ed.). ACM, 906–908. doi:10.1145/1066157.1066283
- [3] Sarah Azzabi, Zakiya Alfughi, and Abdelkader Ouda. 2024. Data Lakes: A Survey of Concepts and Architectures. *Comput.* 13, 7 (2024), 183. doi:10.3390/COMPUTERS13070183
- [4] Zhenbiao Cao, Yuanlei Zheng, Zhihao Fan, Xiaojin Zhang, Wei Chen, and Xiang Bai. 2024. RSL-SQL: Robust Schema Linking in Text-to-SQL Generation. *CoRR* abs/2411.00073 (2024). arXiv:2411.00073 doi:10.48550/ARXIV.2411.00073
- [5] Shiri Chechik, Michael Langberg, David Peleg, and Liam Roditty. 2009. Fault-tolerant spanners for general graphs. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 435–444.
- [6] Shiri Chechik, Michael Langberg, David Peleg, and Liam Roditty. 2009. Fault-tolerant spanners for general graphs. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 435–444.
- [7] City of New York. n.d.. NYC Open Data. <https://opendata.cityofnewyork.us/>. Accessed: 2026-03-01.
- [8] Arash Dargahi Nobari and Davood Rafiei. 2024. Dtt: An example-driven tabular transformer for joinability by leveraging large language models. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–24.
- [9] dbt Labs. 2025. dbt. <https://www.getdbt.com/> Website, accessed Nov. 1, 2025.
- [10] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Format Restriction, and Column Exploration. *CoRR* abs/2502.00675 (2025). arXiv:2502.00675 doi:10.48550/ARXIV.2502.00675
- [11] Yuyang Dong, Kunihiro Takeoka, Chuan Xiao, and Masafumi Oyamada. 2021. Efficient joinable table discovery in data lakes: A high-dimensional similarity-based approach. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 456–467.
- [12] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2022. Deepjoin: Joinable table discovery with pre-trained language models. *arXiv preprint arXiv:2212.07588* (2022).
- [13] EcoTable Contributors. 2025. EcoTable: Ad. <https://github.com/yuhuiwang02/EcoTable/ad>. GitHub repository, accessed Nov. 1, 2025.

- [14] EcoTable Contributors. 2025. EcoTable: Business. <https://github.com/yuhuiwang02/EcoTable/business>. GitHub repository, accessed Nov. 1, 2025.
- [15] EcoTable Contributors. 2025. EcoTable: Engagement. <https://github.com/yuhuiwang02/EcoTable/engagement>. GitHub repository, accessed Nov. 1, 2025.
- [16] EcoTable Contributors. 2025. EcoTable: NYC. <https://github.com/yuhuiwang02/EcoTable/NYC>. GitHub repository, accessed Nov. 1, 2025.
- [17] EcoTable Contributors. 2025. EcoTable: Platform. <https://github.com/yuhuiwang02/EcoTable/platform>. GitHub repository, accessed Nov. 1, 2025.
- [18] Meihao Fan, Ju Fan, Nan Tang, Lei Cao, Guoliang Li, and Xiaoyong Du. 2024. Autoprep: Natural language question-aware data preparation with a multi-agent framework. *arXiv preprint arXiv:2412.10422* (2024).
- [19] Meihao Fan, Xiaoyue Han, Ju Fan, Chengliang Chai, Nan Tang, Guoliang Li, and Xiaoyong Du. 2024. Cost-Effective In-Context Learning for Entity Resolution: A Design Space Exploration. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 3696–3709. doi:10.1109/ICDE60146.2024.00284
- [20] Naman Goyal, Jingfei Du, Myle Ott, Giri Anantharaman, and Alexis Conneau. 2021. Larger-scale transformers for multilingual masked language modeling. *arXiv preprint arXiv:2105.00572* (2021).
- [21] Rihan Hai, Christos Koutras, Christoph Quix, and Matthias Jarke. 2023. Data lakes: A survey of functions and systems. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2023), 12571–12590.
- [22] Benjamin Hättasch, Michael Truong-Ngoc, Andreas Schmidt, and Carsten Binnig. 2020. It's AI Match: A Two-Step Approach for Schema Matching Using Embeddings. In *AIDB@VLDB 2020, 2nd International Workshop on Applied AI for Database Systems and Applications, Held with VLDB 2020, Monday, August 31, 2020, Online Event / Tokyo, Japan*, Bingsheng He, Berthold Reinwald, and Yingjun Wu (Eds.). https://drive.google.com/file/d/1xCGhHfghJR3DLuA-9oXQXB58M1_8a4P/view?usp=sharing
- [23] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTaV3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543* (2021).
- [24] Madelon Hulsebos, Kevin Hu, Michiel Bakker, Emanuel Zraggen, Arvind Satyanarayan, Tim Kraska, Çağatay Demiralp, and César Hidalgo. 2019. Sherlock: A deep learning approach to semantic data type detection. In *Proceedings of the 25th ACM SIGKDD International Conference on knowledge discovery & data mining*. 1500–1508.
- [25] Aamod Khatiwada, Roece Shraga, and Renée J. Miller. 2026. Fuzzy Integration of Data Lake Tables. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026*, Wolfgang Lehner, Vanessa Braganholo, Kostas Stefanidis, Zheyang Zhang, Alexander Krause, and João Felipe Nicolaci Pimentel (Eds.). OpenProceedings.org, 96–102. doi:10.48786/EDBT.2026.08
- [26] Dénes König. 1916. Über graphen und ihre anwendung auf determinantentheorie und mengenlehre. *Math. Ann.* 77, 4 (1916), 453–465.
- [27] Lawrence Kou, George Markowsky, and Leonard Berman. 1981. A fast algorithm for Steiner trees. *Acta informatica* 15, 2 (1981), 141–145.
- [28] Christos Koutras, Jiani Zhang, Xiao Qin, Chuan Lei, Vasileios Ioannidis, Christos Faloutsos, George Karypis, and Asterios Katsifodimos. 2024. OmniMatch: Effective self-supervised any-join discovery in tabular data repositories. *arXiv preprint arXiv:2403.07653* (2024).
- [29] Eugenie Lai, Yeye He, and Surajit Chaudhuri. 2025. Auto-Prep: Holistic Prediction of Data Preparation Steps for Self-Service Business Intelligence. *Proc. VLDB Endow.* 18, 7 (2025), 2212–2225. <https://www.vldb.org/pvldb/vol18/p2212-he.pdf>
- [30] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. 2024. Mcs-sql: Leveraging multiple prompts and multiple-choice selection for text-to-sql generation. *arXiv preprint arXiv:2405.07467* (2024).
- [31] Jihyung Lee, Jin-Seop Lee, Jaehoon Lee, YunSeok Choi, and Jee-Hyong Lee. 2025. DCG-SQL: Enhancing In-Context Learning for Text-to-SQL with Deep Contextual Schema Link Graph. *arXiv preprint arXiv:2505.19956* (2025).
- [32] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763* (2024).
- [33] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, et al. 2025. Omnisql: Synthesizing high-quality text-to-sql data at scale. *arXiv preprint arXiv:2503.02240* (2025).
- [34] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13067–13075.
- [35] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
- [36] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.
- [37] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. 2023. Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples. *Proc. VLDB Endow.* 16, 11 (2023), 3391–3403. doi:10.14778/3611479.3611534
- [38] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2024. Table-gpt: Table fine-tuned gpt for diverse table tasks. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
- [39] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* 14, 1 (2020), 50–60. doi:10.14778/3421424.3421431
- [40] Yiming Lin, Yeye He, and Surajit Chaudhuri. 2023. Auto-bi: Automatically build bi-models leveraging local join prediction and global schema graph. *arXiv preprint arXiv:2306.12515* (2023).
- [41] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR abs/1907.11692* (2019). arXiv:1907.11692 <http://arxiv.org/abs/1907.11692>
- [42] Yurong Liu, Eduardo Peña, Aécio S. R. Santos, Eden Wu, and Juliana Freire. 2025. Magnet: Combining Small and Large Language Models for Schema Matching. *Proc. VLDB Endow.* 18, 8 (2025), 2681–2694. doi:10.14778/3742728.3742757
- [43] Arash Dargahi Nobari and Davood Rafiei. 2022. Efficiently Transforming Tables for Joinability. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 1649–1661. doi:10.1109/ICDE53745.2022.00169
- [44] Arash Dargahi Nobari and Davood Rafiei. 2025. TabulaX: Leveraging Large Language Models for Multi-Class Table Transformations. *Proc. VLDB Endow.* 18, 11 (2025), 3826–3839. <https://www.vldb.org/pvldb/vol18/p3826-nobari.pdf>
- [45] OpenAI n.d.. OpenAI API Pricing. <https://openai.com/api/pricing>. [n.d.] Website, accessed Nov. 1, 2025.
- [46] George Papadakis, Ekaterini Ioannou, Emanouil Thanos, and Themis Palpanas. 2021. *The Four Generations of Entity Resolution*. Morgan & Claypool Publishers. doi:10.2200/S01067ED1V01Y202012DTM064
- [47] Marcel Parciak, Brecht Vandevort, Frank Neven, Liesbet M. Peeters, and Stijn Vansummen. 2025. LLM-Matcher: A Name-Based Schema Matching Tool using Large Language Models. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*, Volker Markl, Joseph M. Hellerstein, and Azza Abouzied (Eds.). ACM, 203–206. doi:10.1145/3722212.3725112
- [48] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2023), 36339–36348.
- [49] Mattia Di Profio, Mingjun Zhong, Yaji Sripada, and Marcel Jaspars. 2025. FlowETL: An Autonomous Example-Driven Pipeline for Data Engineering. *CoRR abs/2507.23118* (2025). arXiv:2507.23118 doi:10.48550/ARXIV.2507.23118
- [50] Torsten Scholach, Nathan Schucher, and Dmity Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv preprint arXiv:2109.05093* (2021).
- [51] Nima Shahbazi, Jin Wang, Zhengjie Miao, and Nikita Bhutani. 2024. Fairness-Aware Data Preparation for Entity Matching. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 3476–3489. doi:10.1109/ICDE60146.2024.00268
- [52] Roece Shraga, Avigdor Gal, and Hagai Roitman. 2020. AdNEV: Cross-Domain Schema Matching using Deep Similarity Matrix Adjustment and Evaluation. *Proc. VLDB Endow.* 13, 9 (2020), 1401–1415. doi:10.14778/3397230.3397237
- [53] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. Mpnnet: Masked and permuted pre-training for language understanding. *Advances in neural information processing systems* 33 (2020), 16857–16867.
- [54] Shayam Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755* (2024).
- [55] The pandas development team. n.d.. Pivot in Python Pandas. <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pivot.html>. Accessed: 2026-02-28.
- [56] The pandas development team. n.d.. Python Series Str.split. <https://pandas.pydata.org/docs/reference/api/pandas.Series.str.split.html>. Accessed: 2026-02-28.
- [57] Vadim G Vizing. 1965. The chromatic class of a multigraph. *Cybernetics* 1, 3 (1965), 32–41.
- [58] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, 540–557. <https://aclanthology.org/2025.coling-main.36/>

- [59] Jin Wang and Yuliang Li. 2022. Minun: evaluating counterfactual explanations for entity matching. In *DEEM '22: Proceedings of the Sixth Workshop on Data Management for End-To-End Machine Learning Philadelphia, PA, USA, 12 June 2022*, Matthias Boehm, Paroma Varma, and Doris Xin (Eds.). ACM, 7:1–7:11. doi:10.1145/3533028.3533304
- [60] Jin Wang, Yuliang Li, and Wataru Hirota. 2021. Machamp: A Generalized Entity Matching Benchmark. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*, Gianluca Demartini, Guido Zuccon, J. Shane Culpepper, Zi Huang, and Hanghang Tong (Eds.). ACM, 4633–4642. doi:10.1145/3459637.3482008
- [61] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, 2609–2634. doi:10.18653/V1/2023.ACL-LONG.147
- [62] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2024. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=4L0xnS4GQM>
- [63] Cong Yan and Yeye He. 2020. Auto-Suggest: Learning-to-Recommend Data Preparation Steps Using Data Science Notebooks. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 1539–1554. doi:10.1145/3318464.3389738
- [64] Junwen Yang, Yeye He, and Surajit Chaudhuri. 2021. Auto-Pipeline: Synthesize Data Pipelines By-Target Using Reinforcement Learning and Search. *Proc. VLDB Endow.* 14, 11 (2021), 2563–2575. doi:10.14778/3476249.3476303
- [65] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X
- [66] Hanchong Zhang, Ruisheng Cao, Lu Chen, Hongshen Xu, and Kai Yu. 2023. ACT-SQL: In-context learning for text-to-SQL with automatically-generated chain-of-thought. *arXiv preprint arXiv:2310.17342* (2023).
- [67] Yong Zhang, Jiacheng Wu, Jin Wang, and Chunxiao Xing. 2020. A Transformation-Based Framework for KNN Set Similarity Search. *IEEE Trans. Knowl. Data Eng.* 32, 3 (2020), 409–423. doi:10.1109/TKDE.2018.2886189
- [68] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J Miller. 2019. Josie: Overlap set similarity search for finding joinable tables in data lakes. In *Proceedings of the 2019 International Conference on Management of Data*. 847–864.
- [69] Erkang Zhu, Yeye He, and Surajit Chaudhuri. 2017. Auto-join: Joining tables by leveraging transformations. *Proceedings of the VLDB Endowment* 10, 10 (2017), 1034–1045.
- [70] Erkang Zhu, Fatemeh Nargesian, Ken Q Pu, and Renée J Miller. 2016. LSH ensemble: Internet-scale domain search. *arXiv preprint arXiv:1603.07410* (2016).

A Training Data Collection For M_S

In the table identification layer, the deep learning model M_S (i.e., RoBERTa) performs coarse-grained filtering to retrieve query-related tables. The training data consists of (NL query, table) pairs, each assigned a binary label $y \in \{0, 1\}$. This label denotes whether the table is relevant ($y = 1$) or irrelevant ($y = 0$) to the NL query.

We construct the initial training dataset through the following three steps: (1) we collect DBT projects and partition them into training and testing subsets; (2) we use LLMs to generate natural language queries from existing SQL in training sets, and then manually verify that each query is correctly aligned with its SQL; (3) we leverage sqlparse to identify tables referenced in the SQL and label them as relevant ($y = 1$), while marking all remaining tables in the same data lake as irrelevant ($y = 0$). To scale this process, we follow OmniSQL [33] by generating a step-by-step Chain-of-Thought (CoT) explanation that describes the reasoning from the natural language query to the SQL code. This CoT serves as an automatic consistency check to guarantee semantic alignment between the query and SQL, eliminating manual verification. Using this procedure, we construct a large-scale training set with 200,000 labeled (query, table) pairs.

B Details of the Table Identification Layer

In the table identification layer, the LLM takes the original NL query and the coarse-grained candidate tables as input, and outputs the final verified table set. To achieve this, the LLM executes two guided operations: “Query Understanding” and “Schema Reasoning and Verification”. The detailed prompts used to guide the LLM in both operations are provided as follows.

Prompt for Table Identification

You are a data lake schema expert specializing in large-scale, open government data lakes. Your role is to identify, from a pool of candidate tables retrieved by a deep learning model, the minimal and semantically correct set of tables required to answer a given natural language query.

You reason at the column level: a table is relevant if and only if its columns directly supply information requested by the query. In this data lake, tables may share naming prefixes and be organized by temporal scope or horizontal data partitioning; your reasoning must account for both aspects.

Step 1: Query Understanding

Given the following natural language query, decompose it into a structured list of atomic semantic components. Each component should represent one self-contained information need, such as:

- A key entity or subject (e.g., “taxi trip”, “building permit”).
- A measurable attribute or metric (e.g., “trip duration”, “fare”).
- A filter or scoping condition (e.g., “yellow cab only”).
- A temporal specification (e.g., “in 2017”, “FY 2018–2019”).

Pay particular attention to temporal references. If the query mentions a specific year, fiscal year, or date range, record it explicitly, as it will be used to resolve temporal ambiguity among candidate tables in the verification step.

Input

Query: query

Output Format (JSON Only)

```
{
  "temporal_scope": "<year / fiscal year / date range,
or null>",
  "semantic_components": [
    "component 1: <description>",
    "component 2: <description>"
    ...
  ]
}
```

Step 2: Schema Reasoning and Alignment

You are given the original query, its decomposed semantic components (including temporal scope), and a ranked list of candidate tables from the data lake. Each table entry includes its name and a sample of its column names.

For each candidate table, reason step-by-step to determine whether the table directly contributes at least one column that satisfies a semantic component of the query. Apply the following principles in order:

- **Column-Level Matching:** A table is relevant only if its columns directly provide data requested by the query. Do not rely on table names alone.
- **Temporal Scope Consistency:** If a temporal scope exists, discard tables whose names encode a different year or fiscal year, even if schemas match.
- **Partition-Complete Selection:** For horizontally partitioned sub-tables (e.g., records, records_1), examine and retain each variant independently if they are individually relevant.
- **Non-Redundancy:** If two tables have identical schemas and partitions, retain only the one with broader coverage or the lower partition index.

Input

Query: {query}

Temporal Scope: {temporal_scope}

Semantic Components: {semantic_components}

Candidate Tables: {candidate_tables_with_columns}

Output Format (JSON Only)

```
{
  "table_reasoning": {
    "<table_name>": {
      "matched_components": ["component i"],
      "matched_columns": ["col_a"],
      "temporal_consistent": true/false,
      "verdict": "keep/discard",
      "reason": "<one-sentence justification>"
    },
    ...
  },
  "selected_tables": ["table_1", "table_2", ...]
}
```

C Details of Pairwise Joinability Estimation

We give details on the deep learning model $M_J(\cdot)$ introduced in Section 4.2. Given a pair of columns C_i from table T_i and C_j from table T_j , the model acts as a binary classifier to predict their joinability probability. Subsequently, this score is used to derive the edge weight $p(e_{ij}) \in (0, 1]$ between the two tables. In EcoTable, this model is a modular component instantiated using three methods: Auto-BI [40], DeepJoin [12], and OmniMatch [28].

Auto-BI [40]. Auto-BI represents each column pair (C_i, C_j) with 24 features, categorized into *Metadata-features* and *Data-features*. We list these features, and precise definitions are in [40].

Metadata-features. These features capture structured information about column pairs, derived from table names and column names. In total, A total of 12 features derived from this metadata are utilized, including: Jaccard_similarity, Jaccard_containment, Edit_distance, Embedding_similarity, Token_count, Char_count, Col_frequency, Col_position, Col_relative_position, Unique_col_position, Table_embedding, and Header_jaccard.

Data-features. 13 features are extracted that relate to the data content of the columns. These features include Max_containment, Min_containment, Left_containment, Right_containment, Value_distinct_ratio, Range_overlap, EMD_score, Value_length, Value_type, Row_cnt, Row_ratio, Col_ratio, and Cell_ratio.

Given the limited information about the original model, we focus on two regression alternatives. We denote the MLP-based implementation as Auto-BI_{MLP} and also introduce a tree-based ensemble variant, referred to as Auto-BI_{XGB}.

- Auto-BI_{MLP}. We employ a four-layer Multi-Layer Perceptron (MLP) as the regression model. Specifically, we concatenate the 24 features into a single vector and feed it into the MLP with hidden sizes [256, 256, 128, 64]. The output layer applies a Sigmoid function to produce a probability between 0 and 1. For our experiments, we train the model by minimizing the binary cross-entropy loss.
- Auto-BI_{XGB}. We use Auto-BI_{XGB}, a variant that relies on Gradient Boosting Decision Trees implemented via XGBClassifier. In our experiments, we adopt this implementation for training and inference, using 100 decision trees.

DeepJoin [12]. DeepJoin leverages a Pre-trained Language Model (PLM) to encode each column as a semantic vector, and then uses cosine similarity to assess whether two columns can be joined. Specifically, for each column of the pair, we present it as a text using the pattern: “*table_title.column_name* contains *n* values (*max_len*, *min_len*, *avg_len*): *cell*₁, *cell*₂, ..., *cell*_{*n*}”. Here, *n* denotes the number of distinct cell values in the column; *max_len*, *min_len*, *avg_len* are the maximum, minimum, and average numbers of words per cell, respectively; and *cell*_{*i*} denotes the *i*-th distinct cell value selected based on frequency. For this text, we then use MPNet [53] to compute column embeddings as input representations. For each pair, we denote the input as (X_k, Y_k) , where *k* indexes the pairs and X_k, Y_k are the embedding vectors for the columns in that pair.

To optimize this PLM, the Multiple Negative Ranking Loss is adopted in DeepJoin, which computes the negative log-likelihood of softmax-normalized scores:

$$\mathcal{L} = -\frac{1}{N} \sum_{k=1}^N \left[S(X_k, Y_k) - \log \sum_{l=1}^N \exp(S(X_k, Y_l)) \right] \quad (8)$$

where $S(\cdot, \cdot)$ denotes the cosine similarity, and *N* is the batch size.

OmniMatch [28]. Unlike the methods above, OmniMatch builds a feature vector for each column by combining two components: 1) representative statistics from that column, following the design in [24], and 2) features learned by applying a GNN to a graph of semantically similar columns, so that information can be shared across similar columns. In more detail, OmniMatch defines five

pairwise column similarities and uses them to construct a global graph where nodes are columns and edges are weighted by similarity scores in (0, 1]. OmniMatch then applies an RGCN on the graph to aggregate information from columns that are similar to a given column. In this way, each column C_i gets a final feature vector h_i that captures these similarity relationships. Finally, to predict the joinability of (C_i, C_j) , we first compute the distance between their feature vectors h_i and h_j , and then apply an exponential decay function $\exp(-\cdot)$, to map this distance to a joinability probability. In our experiments, we train the model parameters using a triplet margin loss.

D Details of LLM-based edge validation.

In this section, we detail priority-aware edge validation and prompt construction.

Priority-aware edge validation. We agree with the reviewer that if the LLM incorrectly invalidates a critical edge that is essential for table connectivity, other edges would be invalid. To mitigate this, EcoTable first identifies *critical* edges as follows: 1) edges that appear in Steiner trees corresponding to multiple queries within the same iteration, and 2) edges whose deletion would prevent the construction of a new Steiner tree for a query in the subsequent iteration. For such a case, EcoTable makes three independent LLM calls to validate it and uses majority vote to make the final decision. This significantly reduces the impact of hallucinations and preserves reliable graph connectivity.

Table Serialization in Prompts. We use a two-phase serialization strategy that we believe addresses the reviewer’s concern. In the first step, we would like to clarify that we do not involve all rows to avoid noise. Instead, we randomly sample about 50 rows respectively from a pair of tables. Then we feed them to the LLM and ask it to identify several candidate column pairs that are likely to be joinable. Subsequently, we only consider the semantic relationships between these pairs in the second step. For each pair of tables, we provide all their distinct values for an in-depth comparison. The LLM then outputs a binary decision (1 for valid, 0 for invalid), indicating whether a join relationship exists. This two-phase approach ensures that we collect sufficient overlapping values to avoid incorrect non-joinable decisions. At the same time, it keeps the prompts compact, thus reducing hallucinations and performance degradation. The prompt is detailed as follows:

Prompt for Join Validation

You are a rigorous data architect. Your task is to analyze two given data tables and determine whether there is a reasonable business connection between them.

Background

The goal of joining tables is to enrich the information used to answer a query. A “valid join” occurs when you combine complementary attributes from different perspectives within the same scope. For example, joining a school’s basic info with its program details for the same year and region allows a query to access both dimensions.

Conversely, tables covering different time periods, regions, or categories are "parallel slices." Joining them does not enrich information; it corrupts the coherence of results by mixing data from incompatible scopes (e.g., mixing a 2019 catalog with a 2022 catalog).

Workflow Explanation

To ensure accuracy, we follow a two-step verification process:

Step 1: Semantic & Scope Alignment

Examine the table names and sampled rows to identify if they share the same temporal, spatial, and categorical scope.

- If scopes differ (e.g., Sales_2021 vs Sales_2023), return an empty selected_pairs.
- If scopes align, identify column pairs that refer to the same specific attribute (e.g., emp_id vs staff_code). Avoid matching different metrics (e.g., max_speed vs avg_speed).

Step 2: Data-Level Validation

For the selected pairs, you will be provided with the full set of distinct values. You must determine if they are:

- Joinable: Columns point to the same real-world entity and values match via exact equality or clear transformation.
- Unjoinable: Irreconcilable differences in values or business context are revealed.

Input Data (Step 1)

Table Names: {TABLE_NAMES}

Table 1 Sample (50 rows): {TABLE1_SAMPLE}

Table 2 Sample (50 rows): {TABLE2_SAMPLE}

Candidate Pairs: {CANDIDATE_LIST}

Output Format (Step 1 - JSON Only)

```
{
  "same_scope": true/false,
  "selected_pairs": ["Table1.ColA <-> Table2.ColB"],
  "reason": "Business logic for semantic matching."
}
```

Input Data (Step 2)

Selected Pairs: {Table1.ColA <-> Table2.ColB}

Table 1 Distinct Values: [val1, val2, ...]

Table 2 Distinct Values: [valA, valB, ...]

Output Format (Step 2 - JSON Only)

```
{
  "can_join": "1/0",
  "alignment_sample": ["val1 <-> valA", "val2 <-> valB"],
  "reason": "Final decision based on value overlap."
}
```

E Comparison of Non-LLM v.s. EcoTable

In this section, we compare EcoTable with a new baseline without involving LLMs, denoted as Non-LLM. This baseline replaces our LLM-based components with traditional deep learning methods: it employs DeBERTa-v3-large for table identification, DeepJoin to predict join paths, and DTT to execute table transformations.

As shown in Figure 17, Non-LLM is more efficient than EcoTable, as it completely avoids the high latency introduced by LLM inference. However, Non-LLM is not effective on complex integration tasks because its components depend on embeddings, which fails to handle the unpredictable noise present in real-world data lakes. In contrast, EcoTable achieves state-of-the-art performance by leveraging the adaptive reasoning and code-generation capabilities

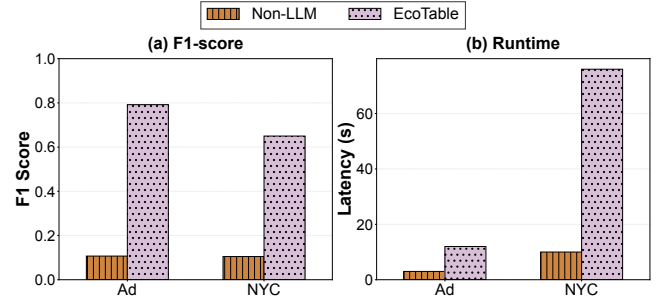


Figure 17: Comparison of Non-LLM v.s. EcoTable .

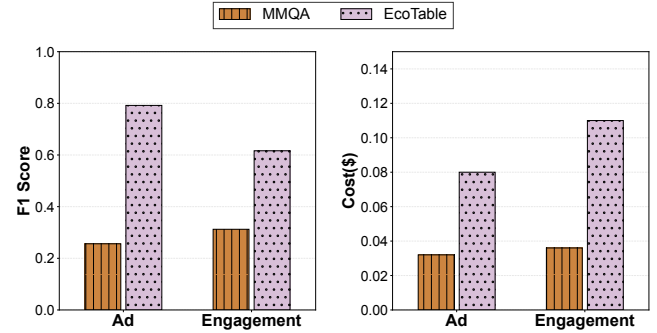


Figure 18: Comparison of MMQA vs EcoTable .

of LLMs. This enables EcoTable to flexibly identify required join paths and resolve complex data inconsistencies during runtime, substantially improving integration quality while only moderately increasing execution time.

F Statistics on Join Types and Transformations

we provide the join and transformation statistics in Table 4.

Table 4: Statistics on Join Types and Transformations.

Dataset	# Equi-joins	# Fuzzy-joins	# Total Transf.
Ad	65	17	17
Engagement	28	10	10
Business	13	7	7
Platform	11	8	8
NYC Data Lake	248	204	2440

G Comparison of MMQA vs EcoTable .

In this section, we incorporate the Multi-Table QA baseline: MMQA replaces our table identification layer with the retriever from MMQA. This method decomposes multi-hop questions into sub-questions and iteratively retrieves tables by jointly considering question-table and table-table relevance.

As illustrated in Figure 18, EcoTable achieves a much higher Join Path F1-score (79.2%) than MMQA (25.62%) while incurring only a modest additional cost because MMQA uses a binary relevance score based on exact matches of column values across tables. Such a rigid criterion performs poorly in heterogeneous data lakes, where

values are often noisy, inconsistent, or partially mismatched. If no exact overlap is found, its iterative retrieval terminates too early. In contrast, EcoTable is more resilient at filtering out irrelevant

tables and preserving accuracy in large-scale settings like the NYC Data Lake. MMQA has a lower cost because it only uses LLMs to decompose queries.